

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ імені ІГОРЯ СІКОРСЬКОГО»
Факультет інформатики та обчислювальної техніки
(повна назва інституту/факультету)

Кафедра інформатики та програмної інженерії
(повна назва кафедри)

«До захисту допущено»

Завідувач кафедри

_____ Едуард ЖАРІКОВ
(підпис) (ім'я прізвище)

“ ” _____ 2022 р.

Дипломний проєкт
на здобуття ступеня бакалавра
за освітньо-професійною програмою «Інженерія програмного забезпечення
комп'ютеризованих систем»
спеціальності «121 Інженерія програмного забезпечення»

на тему: Агрегатор чат-ботів для оптимізації комунікації з клієнтами з
платформи CRM

Виконав студент IV курсу, групи ІП-82
(шифр групи)
Зозуля Ірина Вячеславівна
(прізвище, ім'я, по батькові) _____ (підпис)

Керівник ст.викладач Ковтунець О. В.
(посада, науковий ступінь, вчене звання, прізвище та ініціали) _____ (підпис)

Консультант
з графічної
документації доцент, к.т.н., доц., Ліщук К. І.
(посада, науковий ступінь, вчене звання, прізвище та ініціали) _____ (підпис)

Рецензент доцент кафедри ІСТ, к.т.н., доц., Остапченко К. Б.
(посада, науковий ступінь, вчене звання, прізвище та ініціали) _____ (підпис)

Засвідчую, що у цій дипломній роботі
немає запозичень з праць інших
авторів без відповідних посилань.

Студент _____
(підпис)

Київ –2022

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Рівень вищої освіти – перший (бакалаврський)

Спеціальність – 121 Інженерія програмного забезпечення

Освітньо-професійна програма – Інженерія програмного забезпечення
комп'ютеризованих систем

ЗАТВЕРДЖУЮ

Завідувач кафедри

(підпис) Едуард ЖАРІКОВ
(ім'я прізвище)

“ ” _____ 2022 р.

ЗАВДАННЯ
на дипломний проєкт студенту

Зозулі Ірині Вячеславівні
(прізвище, ім'я, по батькові)

1. Тема проєкту Агрегатор чат-ботів для оптимізації комунікації з клієнтам
з платформи CRM

керівник проєкту Ковтунець Олесь Володимирович, ст.викладач
(прізвище, ім'я, по батькові, науковий ступінь, вчене звання)

затверджені наказом по університету від «10»квітня 2022 р. № 1033-с

2. Термін подання студентом проєкту « 19 » червня 2022 року

3. Вихідні дані до проєкту: технічне завдання

4. Зміст пояснювальної записки

1) Загальні положення: основні визначення та терміни, опис предметного
середовища, огляд ринку програмних продуктів, постановка задачі.

2) Інформаційне забезпечення: вхідні дані, вихідні дані, опис структури бази
даних.

3) Математичне забезпечення: змістовна та математична постановки задачі,
обґрунтування та опис методу розв'язання.

4) Програмне та технічне забезпечення: засоби розробки, вимоги до технічного
забезпечення, архітектура програмного забезпечення, побудова звітів.

5) Технологічний розділ: керівництво користувача, методика випробувань
програмного продукту.

5. Перелік графічного матеріалу

1) Схема структурна варіантів використань

2) Схема бази даних

3) Схема структурна класів програмного забезпечення

6. Консультанти розділів проєкту

Розділ	Прізвище, ініціали та посада консультанта	Підпис, дата	
		завдання видав	завдання прийняв

7. Дата видачі завдання «10» березня 2022 року

Календарний план

№ з/п	Назва етапів виконання дипломного проєкту	Термін виконання етапів проєкту	Примітка
1	Вивчення рекомендованої літератури	10.03.2022	
2	Аналіз існуючих методів розв'язання задачі	17.03.2022	
3	Постановка та формалізація задачі	27.03.2022	
4	Розробка інформаційного забезпечення	08.04.2022	
5	Алгоритмізація задачі	19.04.2022	
6	Обґрунтування вибору використаних технічних засобів	27.04.2022	
7	Розробка програмного забезпечення	01.05.2022	
8	Налагодження програми	17.05.2022	
9	Виконання графічних документів	17.03.2022	
10	Оформлення пояснювальної записки	17.05.2022	
11	Подання ДП на попередній захист	06.06.2022	
12	Подання ДП рецензенту	12.06.2022	
13	Подання ДП на основний захист	19.06.2022	

Студент

(підпис)

Ірина ЗОЗУЛЯ

(ініціали, прізвище)

Керівник

(підпис)

Олесь КОВТУНЕЦЬ

(ініціали, прізвище)

АНОТАЦІЯ

Пояснювальна записка дипломного проєкту складається з чотирьох розділів, містить 19 таблиць, 22 рисунків та 6 джерел – загалом 50 сторінок.

Дипломний проєкт присвячений використанню чат-ботів для комерційних цілей та їхній інтеграції у функціонал платформи CRM.

Метою є створити агрегатор чат-ботів різних месенджерів для оптимізації комунікації з клієнтами з платформи CRM.

Об'єкт дослідження: програмне забезпечення для роботи з чат-ботами.

Предмет дослідження: програмне забезпечення для комунікації через чат-боти та обробка засобами платформи CRM результатів комунікації.

У першому розділі виконано аналіз предметної області, відомих технічних рішень, сформульовано функціональні та нефункціональні вимоги до розроблюваного програмного забезпечення.

Другий розділ присвячений моделюванню та проектуванню структури програмного продукту.

У третьому розділі описано планування та виконання тестування розробленого програмного забезпечення.

У четвертому розділі описано керівництво користувача.

КЛЮЧОВІ СЛОВА: ЧАТ-БОТ, CUSTOMER RELATIONSHIP MANAGEMENT, ENTERPRISE RESOURCE PLANNING, ЛІД, УГОДА, БАЗА ДАНИХ.

ABSTRACT

The explanatory note of the diploma project consists of four sections, contains 19 tables, 22 figures and 6 sources – in total 50 pages.

The diploma project is dedicated to the use of chatbots for commercial purposes and their integration into the functionality of the CRM platform.

The purpose of the diploma project is to create an aggregator of chatbots of different messengers to optimize communication with customers from the CRM platform.

Object of research: software for working with chatbots.

Subject of research: software for communication via chatbots and processing of communication results by means of CRM platform.

The first section analyzes the subject area, known technical solutions, formulates functional and non-functional requirements for the developed software.

The second section is devoted to modeling and designing the structure of the software product.

The third section describes the planning and testing of the developed software.

The fourth section describes the user guide.

KEYWORDS: CHATBOT, CUSTOMER RELATIONSHIP MANAGEMENT, ENTERPRISE RESOURCE PLANNING, LEAD, SALE, DATABASE.

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2022 р.

**АГРЕГАТОР ЧАТ-БОТІВ ДЛЯ ОПТИМІЗАЦІЇ КОМУНІКАЦІЇ З
КЛІЄНТАМИ З ПЛАТФОРМИ CRM**

Технічне завдання

КПІ.ІІ-8215.045430.01.91

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олесь КОВТУНЕЦЬ

Нормоконтроль:

_____ Катерина ЛІЩУК

Виконавець:

_____ Ірина ЗОЗУЛЯ

Київ – 2022

ЗМІСТ

1 НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ	3
2 ПІДСТАВА ДЛЯ РОЗРОБКИ	4
3 ПРИЗНАЧЕННЯ РОЗРОБКИ	5
4 ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ	6
4.1 Вимоги до функціональних характеристик.....	6
4.2 Вимоги до надійності.....	6
4.3 Умови експлуатації	6
4.4 Вимоги до складу і параметрів технічних засобів.....	7
4.5 Вимоги до інформаційної та програмної сумісності.....	7
4.6 Вимоги до маркування та пакування	7
4.7 Вимоги до транспортування та зберігання.....	7
4.8 Спеціальні вимоги.....	7
5 ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ.....	9
6 СТАДІЇ І ЕТАПИ РОЗРОБКИ	11
7 ПОРЯДОК КОНТРОЛЮ ТА ПРИЙМАННЯ.....	12

1 НАЙМЕНУВАННЯ ТА ГАЛУЗЬ ЗАСТОСУВАННЯ

Назва розробки: Агрегатор чат-ботів для оптимізації комунікації з клієнтами з платформи CRM.

Галузь застосування:

Наведене технічне завдання поширюється на розробку програмного забезпечення агрегатора чат-ботів, котре використовується для оптимізації комунікації з клієнтами та призначена для користувачів платформи CRM.

					КП.ІП-8215.045430.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		3

2 ПІДСТАВА ДЛЯ РОЗРОБКИ

Підставою для розробки агрегатора чат-ботів для оптимізації комунікації з клієнтами з платформи CRM є завдання на дипломне проєктування, затверджене кафедрою інформатики та програмної інженерії Національного технічного університету України «Київський політехнічний інститут імені Ігоря Сікорського».

					КПІ.ІП-8215.04.54.30.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		4

3 ПРИЗНАЧЕННЯ РОЗРОБКИ

Розробка призначена для використання чат-ботів для комерційних цілей та їхньої інтеграції у функціонал платформи CRM.

Метою розробки є створити агрегатор чат-ботів різних месенджерів для оптимізації комунікації з клієнтами з платформи CRM.

					КП.ІІП-8215.0454.30.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		5

4 ВИМОГИ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1. Вимоги до функціональних характеристик

4.1.1. Програмне забезпечення повинно забезпечувати виконання наступних основних функцій:

4.1.1.1. Для користувача:

- надіслати повідомлення;
- надіслати вкладення;
- прив'язати розмову до ліда;
- прив'язати розмову до контрагента.

4.1.1.2. Для користувача:

- налаштування параметрів агрегатора;
- налаштування чат-ботів.

4.1.2. Розробку виконати на платформі .Net

4.1.3. Додаткові вимоги:

Не висуваються.

4.2. Вимоги до надійності

4.2.1. Передбачити контроль введення інформації.

4.2.2. Передбачити захист від некоректних дій користувача.

4.2.3. Забезпечити цілісність інформації в базі даних.

4.3. Умови експлуатації

4.3.1. Умови експлуатації згідно СанПін 2.2.2.542 – 96.

Не висуваються.

4.3.2. Обслуговування та обслуговуючий персонал.

Не висуваються.

					КПІ.ІП-8215.04.54.30.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		6

4.4. Вимоги до складу і параметрів технічних засобів

4.4.1. Програмне забезпечення повинно функціонувати на обчислювальних машинах з ОС macOS.

4.4.2. Мінімальна конфігурація технічних засобів:

4.4.3. Тип процесору

4.4.4. Будь-який процесор що підтримує ОС macOS.

4.4.5. Об'єм ОЗП

4.4.6. 200 Мб, або більше.

4.4.7. Пристрої введення

4.4.8. Оптичний та потоковий пристрої введення.

4.5. Вимоги до інформаційної та програмної сумісності

Програмне забезпечення повинно працювати під управлінням операційної системи macOS.

4.6. Вимоги до маркування та пакування

Вимоги до маркування та пакування не висуваються.

4.7. Вимоги до транспортування та зберігання

Вимоги до транспортування та зберігання не висуваються.

4.8. Спеціальні вимоги

Згенерувати установчу версію програмного забезпечення.

					КПІ.ІП-8215.04.54.30.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		7

5 ВИМОГИ ДО ПРОГРАМНОЇ ДОКУМЕНТАЦІЇ

5.1. Програмні модулі, котрі розробляються, повинні бути задокументовані, тобто тексти програм повинні містити всі необхідні коментарі

5.2. Програмне забезпечення повинно мати довідникову систему

5.3. У склад супроводжувальної документації повинні входити наступні документи:

5.3.1. Пояснювальна записка не менше ніж на 100 аркушах формату А4 (без додатків 5.3.2 - 5.3.6).

5.3.2. Технічне завдання

5.3.3. Керівництво користувача

5.3.4. Програма та методика тестування

5.4. Графічна частина повинна бути виконана на 3 листах формату А3, котрі включаються у якості графічних матеріалів до пояснювальної записки:

5.4.1. Схема структурна варіантів використання

5.4.2. Схема бази даних

5.4.3. Схема структурна класів програмного забезпечення

					КПІ.ІП-8215.04.54.30.01.91	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		8

6 СТАДІЇ І ЕТАПИ РОЗРОБКИ

№	Назва етапу	Строк	Звітність
1.	Вивчення рекомендованої літератури	10.03.2022	
2.	Аналіз існуючих методів розв'язання задачі	17.03.2022	Пояснювальна записка
3.	Постановка та формалізація задачі	27.03.2022	Технічне завдання
4.	Розробка інформаційного забезпечення	08.04.2022	Специфікації програмного забезпечення
5.	Алгоритмізація задачі	19.04.2022	Схема структурна програмного забезпечення та специфікація компонентів (діаграма класів, схема алгоритму)
6.	Обґрунтування вибору використаних технічних засобів	27.04.2022	Пояснювальна записка
7.	Розробка програмного забезпечення	01.05.2022	Тексти програмного забезпечення
8.	Налагодження програми	17.05.2022	Тести, результати тестування
9.	Виконання графічних документів	17.03.2022	Графічний матеріал проекту
10.	Оформлення пояснювальної записки	17.05.2022	Пояснювальна записка
11.	Подання ДП на попередній захист	06.06.2022	
12.	Подання ДП рецензенту	12.06.2022	
13.	Подання ДП на основний захист	19.06.2022	

7 ПОРЯДОК КОНТРОЛЮ ТА ПРИЙМАННЯ

Тестування розробленого програмного продукту описано в пояснювальній записці «КП.ІП-8215.045430.02.81», розділі 3 «Аналіз якості та тестування програмного забезпечення».

					КП.ІП-8215.045430.01.91	Арк.
						10
Змін.	Арк.	№ докум.	Підп.	Дата.		

**Пояснювальна записка
до дипломного проєкту**

на тему: Агрегатор чат-ботів для оптимізації комунікації з клієнтами з
платформи CRM

КПІ.ІП-8215.045430.02.81

Київ – 2022

ЗМІСТ

Вступ.....	9
1 АНАЛІЗ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.....	10
1.1 Загальні положення.....	10
1.2 Змістовний опис і аналіз предметної області.....	11
1.2.1 ERP-системи	11
1.2.2 Структура CRM	12
1.2.3 Чат-боти.....	14
1.3 Аналіз успішних ІТ-проектів	15
1.3.1 Аналіз відомих технічних рішень.....	15
1.3.2 Аналіз відомих програмних продуктів	17
1.4 Аналіз вимог до програмного забезпечення	18
1.4.1 Розроблення функціональних вимог.....	18
1.4.2 Розроблення нефункціональних вимог.....	19
1.4.3 Постановка комплексу завдань модулю	19
Висновки до розділу	20
2 МОДЕЛЮВАННЯ ТА КОНСТРУЮВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.....	22
2.1 Моделювання та аналіз програмного забезпечення.....	22
2.2 Архітектура програмного забезпечення	23
2.3 Проектування бази даних	29
2.4 Аналіз безпеки даних.....	30
Висновки до розділу	30
3 АНАЛІЗ ЯКОСТІ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.....	31

3.1	Аналіз якості ПЗ.....	31
3.2	Опис процесів тестування.....	31
3.3	Опис контрольного прикладу	33
	Висновки до розділу	35
4	ВПРОВАДЖЕННЯ ТА СУПРОВІД ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ.....	36
4.1	Розгортання програмного забезпечення.....	36
4.2	Робота з програмним забезпеченням	36
	Висновки до розділу	47
	ВИСНОВКИ.....	49
	СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ.....	50
	ДОДАТКИ.....	51

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ

ERP-система	– Enterprise Resource Planning System – система планування ресурсів підприємства.
API	– Application Programming Interface – прикладний програмний інтерфейс
IT	– Інформаційні технології
CRM	– Customer relationship management – управління відносинами з клієнтами
БД	– База даних.

					КП.ІП-8215.045430.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		8

ВСТУП

Для сучасних підприємств сьогодні важливо не лише виробляти якісний продукт, а й правильно продавати його. Основою ефективних продажів є взаємодія з клієнтом від його знайомства з продуктом до кінцевої угоди.

Перші спроби створення систем для ведення бізнесу у сучасному розумінні були у 1990-тих роках. Пізніше в зв'язку із «Проблемою 2000 року», також відомою як «Проблема Y2K», багато компаній вирішили переходити зі старих систем саме на ERP. Завдяки також росту популярності ці системи почали стрімко розвиватися, у результаті чого їхні задачі почали ділити на різні функціональні зони, зокрема, CRM.

Від появи ліда до продажу необхідно підтримувати зв'язок з клієнтом та знайомити його з усіма перевагами саме вашого продукту, що є базовим для менеджменту та маркетингу. У цьому випадку бізнесу потрібно підлаштовуватись під клієнта, для чого на допомогу приходять соціальні мережі та, зокрема, месенджери. У такому разі для менеджерів підприємства з'являється проблема у тому, що одночасно потрібно моніторити кілька месенджерів та вручну співставляти листування з лідами в системі. Отже, бажано підвищити продуктивність менеджерів, а саме автоматизувати та оптимізувати комунікацію з клієнтами в різних месенджерах.

Таким чином, створення агрегатора чат-ботів для зручнішого ведення лідів зумовлює актуальність даної роботи.

Об'єкт дослідження даної роботи – програмне забезпечення для роботи з чат-ботами.

Метою даного проєкту є створити агрегатор чат-ботів різних месенджерів для оптимізації комунікації з клієнтами з платформи CRM.

					КПІ.ІП-8215.0454.30.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		9

1 АНАЛІЗ ВИМОГ ДО ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

1.1 Загальні положення

Ефективна робота будь-якого підприємства напряду залежить від швидкої обробки та оптимального зберігання даних, що використовуються на різних етапах бізнес-процесів. Саме для таких цілей вперше почали впроваджувати системи планування ресурсів підприємства (англ. Enterprise resource planning (ERP), укр. і-ар-пі).

Внаслідок подальшого розширення потреб підприємств, які повинні були вирішувати ERP-системи, їх було поділено на функціональні зони. Однією з таких зон є управління відносинами з клієнтами (англ. Customer relationship management (CRM), укр. сі-ар-ем).

Основними проблемами, які покриває модуль CRM є:

- зберігання та обробка даних клієнтів;
- управління продажами;
- планування продажів та маркетингових заходів;
- автоматизований документообіг;
- ведення угод на усіх стадіях бізнес-процесів;
- автоматизація керування робочим часом менеджерів;
- аналіз ефективності маркетингу;
- аналіз ключових показників по результатах роботи менеджерів.

Оскільки на сьогоднішній день для ефективного ведення клієнта від першого контакту до закриття угоди необхідно мати повне уявлення про його бажання і потреби, то це означає, що менеджеру потрібно зібрати та систематизувати достатньо багато інформації про клієнта. Саме тому оптимальна та якісна обробка великих обсягів даних є однією із ключових потреб CRM-систем.

1.2 Змістовний опис і аналіз предметної області

1.2.1 ERP-системи

ERP-система – це тип програмного забезпечення, яке організації використовують для управління повсякденною діловою діяльністю, як-от бухгалтерський облік, закупівлі, управління проєктами, управління ризиками, а також керування ланцюгами поставок.[1]

Системи ERP об'єднують безліч бізнес-процесів і забезпечують потік даних між ними. Збираючи спільні транзакційні дані організації з кількох джерел, системи ERP усувають дублювання даних і забезпечують цілісність даних за допомогою єдиного джерела коректної інформації. Сьогодні системи ERP мають вирішальне значення для управління тисячами підприємств усіх розмірів і в усіх галузях.

Розвиток систем ERP зумовлений початком четвертої промислової революції, також відомої як Industry 4.0. Вона характеризується активним впровадженням роботизованих систем, автоматизацією виробництва, а також Інтернету речей. Саме потреба у ефективному керуванні такими підприємствами спричинила активний попит на системи, що дозволяють отримувати, систематизувати та аналізувати дані співробітниками.

Як було зазначено раніше, ERP-системи поділяються на функціональні зони, які можуть впроваджуватись на підприємство незалежно один від одного, проте найчастіше бізнес застосовує кілька одночасно. У багатьох системах такі зони групують у ERP-модулі, особливістю яких є швидке та зручне розгортання.

Структура базових модулів ERP-систем:

- управління ланцюгом поставок (англ. Supply chain management (SCM));
- управління відносинами з постачальниками (англ. Supplier relationship management (SRM));
- управління персоналом (англ. Human resources (HR));

- управління відносинами з клієнтами (CRM);
- управління життєвим циклом продукції (англ. Product lifecycle management (PLM));
- управління активами підприємства (англ. Enterprise asset management (EAM));
- бухгалтерський облік;
- управління продажами;
- облік.[2]

1.2.2 Структура CRM

CRM – це система, яка одночасно зміцнює відносини з клієнтами, знижуючи витрати і підвищуючи продуктивність і прибутковість бізнесу. Є багато визначень CRM, наведених різними авторами у своїй літературі; деякі трактують CRM як філософію ведення бізнесу, деякі визначають CRM як стратегію, тоді як інші розуміють CRM як інструмент управління відносинами з клієнтами.[3]

Управління відносинами з клієнтами – модуль ERP, що дозволяє автоматизувати ведення ділових стосунків із клієнтами не тільки на рівні обробки контактів із ними або продажів, а й на рівні передбачення та аналіз їхніх потреб для подальшого удосконалення маркетингу та діяльності підприємства загалом.

Цей модуль допомагає компаніям зробити потреби клієнтів зрозумілими, що дає їм змогу підтримувати ці відносини більш розумно і передбачати майбутні запити клієнта. Ці знання забезпечують конкурентоспроможну ключову перевагу для компаній, щоб досягти частки ринку та знизити операційні витрати на утримання своїх клієнтів.[4]

Для простої та оптимальної роботи із даними клієнтів використовують базові типи сутностей:

- лід;

					КПІ.ІП-8215.0454.30.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата		12

- контрагент;
- контактна особа;
- угода.

Початковою точкою у процесі продажу є лід – потенційний клієнт, що не має повністю сформованого запиту або потреби у послугі чи продукції підприємства. Щоб лід був зацікавлений у сформуванні угоди, необхідно збирати та обробляти дані про нього, аналізуючи та пропонуючи йому актуальні для нього послуги. Для цього в системах CRM застосовуються бізнес-процеси – попередньо спрогнозований та заданий порядок обробки ліда, угоди чи документа.

Контрагентом вважається юридично підтверджена одиниця, з якою підприємство веде офіційну співпрацю у вигляді договорів, документації та рахунків. Для співпраці із контрагентами в CRM використовують контактні особи.

Після того, як лід виявив готовність замовити товар чи послугу, він переходить в угоду. На етапі угоди менеджери повністю опрацьовують потреби клієнта та формують відповідну документацію для офіційного затвердження покупки. Саме для цього перед передачею ліда в угоду він обов’язково повинен пройти процес кваліфікації – вивірка юридичних даних та прив’язка ліда до контрагента. Лише з цього моменту лід стає суб’єктом офіційного документообігу.

Після закриття угоди робота із клієнтом не завершується, адже підприємство потенційно може продовжити співпрацю з ним, а тому задачею менеджера є подальша підтримка контакту та збір даних про його потреби.

Оскільки процес продажу від самого початку є складною процедурою, то модуль CRM є якнайкращим інструментом для його автоматизації.

					КПІ.ІП-8215.0454.30.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		13

1.2.3 Чат-боти

Чат-бот – це програмне забезпечення, що застосовується для автоматизації ведення розмови за допомогою тексту або синтезованого мовлення (переведення тексту у аудіоформат). Такий підхід використовується для мінімізації потреби у втручанні людини-агента в розмову.

Ця технологія сьогодні є широко вживаною саме для потреб підприємств, оскільки дозволяє підтримувати контакт із клієнтами до безпосереднього підключення менеджера в розмову. Таким чином можна надати можливість отримувати основну інформацію від чат-бота, а також залучати людські ресурси тільки у разі неспроможності чат-бота надати консультацію самостійно.

Чат-ботів використовують в мобільних додатках або на корпоративних сайтах як онлайн-консультантів. Після сплеску популярності месенджерів, їхні розробники впровадили можливість розробки чат-ботів і на базах їхніх платформ, для чого були розроблені спеціальні прикладні програмні інтерфейси (англ. Application Programming Interface (API), укр. ей-пі-ай).

Таблиця 1.1 – API для чат-ботів основних месенджерів

Назва месенджера	Назви API
Telegram	Bot API
Viber	Viber Node.JS Bot API, Viber REST API, Viber Python Bot API
WhatsApp	WhatsApp Business API, Chat API
Messenger (Facebook)	Send API

Часто чат-боти розробляються на основі технологій машинного навчання та нейронних мереж, проте для виконання даної дипломної роботи будуть використані значно простіший тип чат-ботів, а саме чат-боти для прийому та передачі повідомлень із месенджерів.

1.3 Аналіз успішних ІТ-проектів

1.3.1 Аналіз відомих технічних рішень

Наразі існує багато технічних засобів, які використовуються для створення модулів CRM. Якщо розглянути представлені на ринку продукти, то можна побачити, що є не лише десктопні застосунки, а й мобільні та веб-застосунки також. Таким чином технічні рішення в уже існуючого програмного забезпечення дуже різняться. Однак, беручи до уваги, що станом на сьогоднішній день найбільшу частку займають саме веб-застосунки, розглянемо саме такий тип програмних продуктів.

Як було зазначено раніше, основою модулю є база даних. Серед існуючих систем керування базами даних (англ. Database Management System (DBMS), укр. СУБД/СКБД) можна виділити найпопулярніші: MySQL, PostgreSQL, SQLite, MongoDB та Microsoft SQL Server.[5]

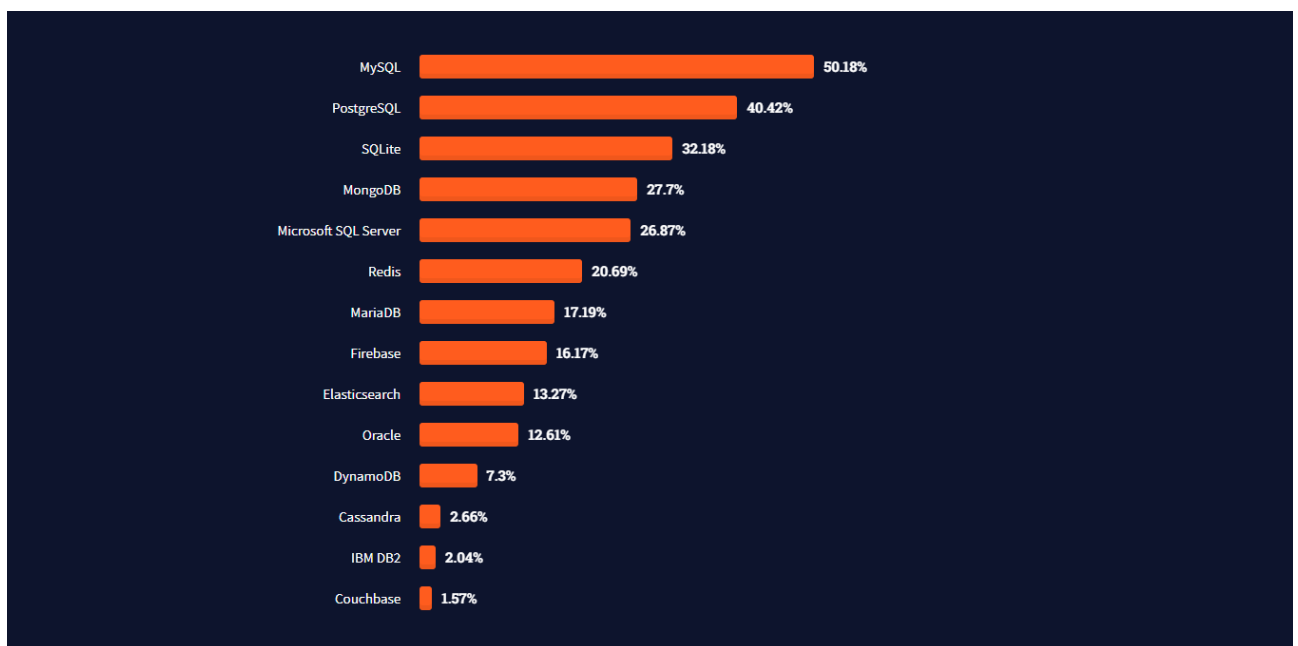


Рисунок 1.1 – Діаграма часток застосування різних СКБД станом на жовтень 2021 року

Варто зазначити, що розподіл, показаний на рисунку 1.1, не варто вважати некоректним, оскільки навіть у межах одного програмного продукту можуть застосовуватись різні системи керування базами даних в залежності від потреб,

які необхідно покрити. Так, бази даних поділяють на реляційні та нереляційні (NoSQL).

До реляційних баз даних належать:

- MariaDB;
- Microsoft SQL Server;
- Oracle;
- PostgreSQL;
- MySQL.

До нереляційних баз даних належать:

- Elasticsearch;
- Firebase;
- MongoDB;
- Redis;
- Cassandra.

Ще однією важливою частиною веб-застосунків є клієнтський інтерфейс.

Для його розробки застосовують такі мови програмування:

- Javascript;
- Python;
- Java;
- PHP;
- Go;
- Kotlin;
- Ruby;
- Rust.

Серед них найпопулярнішою є мова програмування JavaScript, яка має багато фреймворків, які зручно використовувати.

					КПІ.ІП-8215.0454.30.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		16

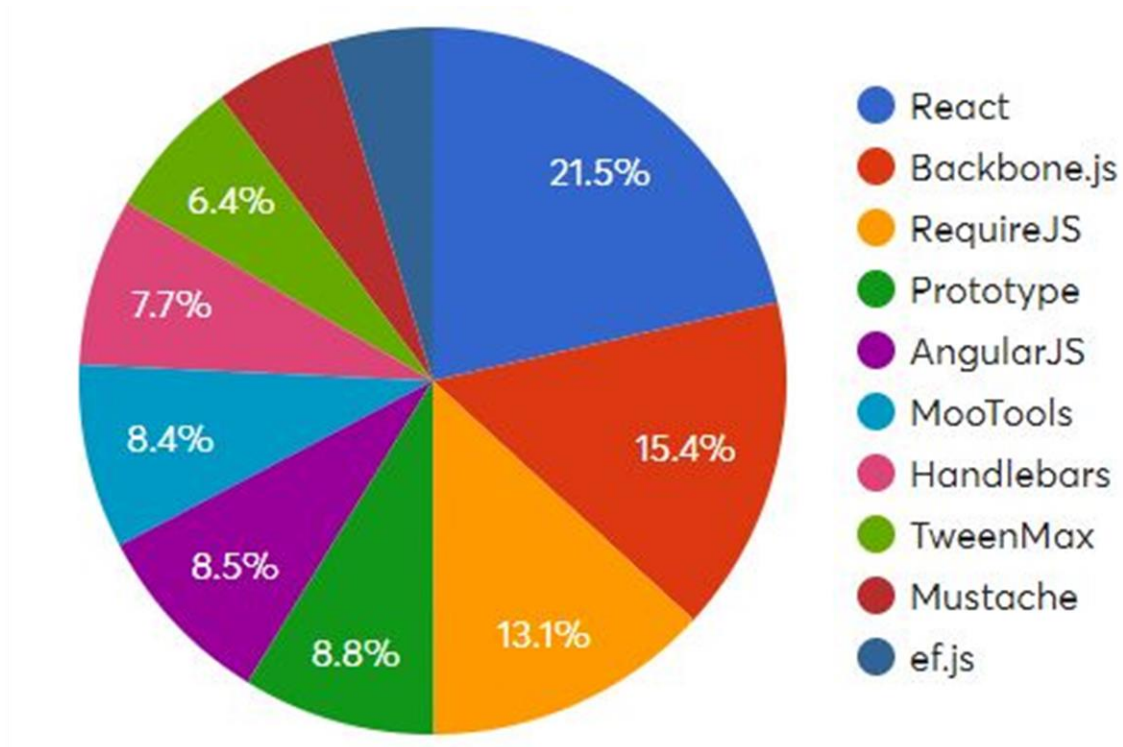


Рисунок 1.2 – Діаграма популярності різних фреймворків JavaScript

Таким чином, завдяки великій кількості технічних рішень, для розробки програмного забезпечення модулю CRM можна легко вибрати зручну для розробника СКБД та мову програмування.

1.3.2 Аналіз відомих програмних продуктів

На українському ринку сьогодні представлені не лише місцеві програмні продукти, а й зарубіжні. Серед відомих українських IT-проектів в сфері ERP-систем з інтеграцією CRM-модулів можна віднести наступні розробки:

- Terrasoft;
- Onebox;
- NetHunt CRM;
- IT-Enterprise;
- SalesDrive.

Найбільш використовуваними станом на 2018 рік можна вважати Бітрікс24, Terrasoft та AmoCRM, які впроваджені у 26%, 15% та 9% опитаних підприємств відповідно.[6]

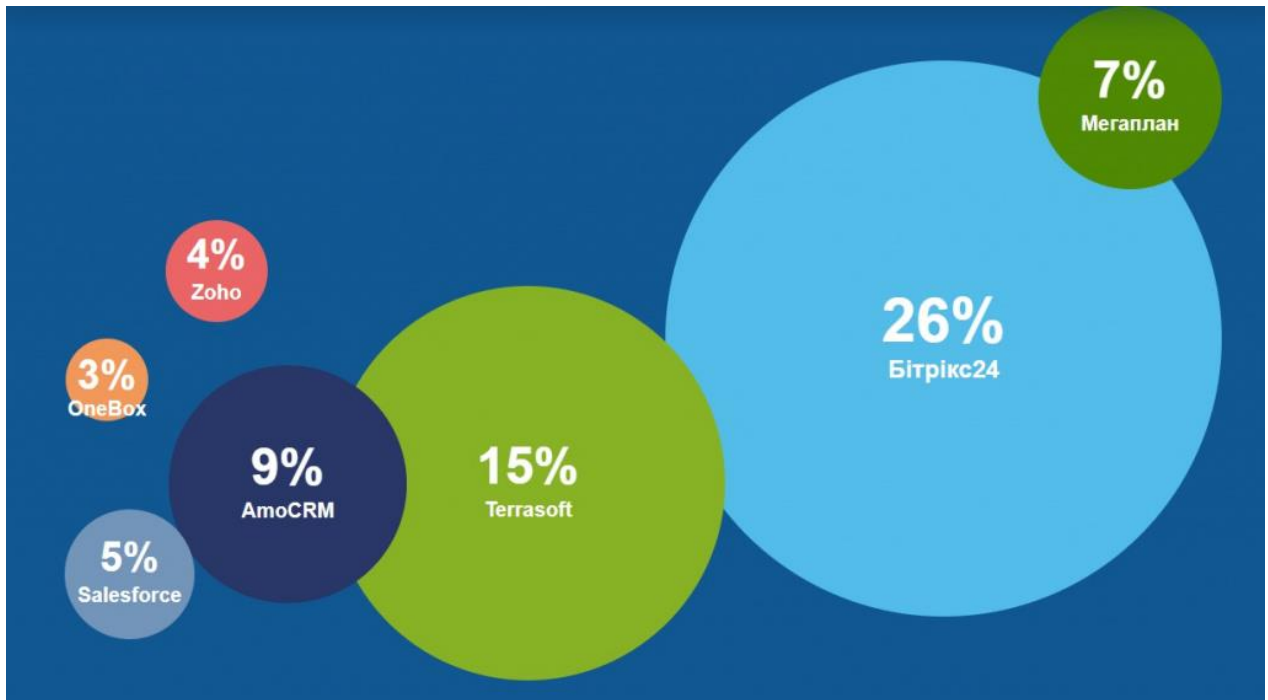


Рисунок 1.3 – Діаграма часток програмних продуктів на українському ринку CRM

1.4 Аналіз вимог до програмного забезпечення

1.4.1 Розроблення функціональних вимог

Для користувача модулю CRM необхідно реалізувати можливість працювати з клієнтами у різних месенджерах через один інтерфейс. Для досягнення цієї цілі цього потрібно виконати такі задачі:

- інтеграція чатів в CRM через API чат-ботів різних месенджерів;
- налаштування збереження результатів контактів з клієнтами у лідах в системі CRM;
- розробити можливість надсилати текстові повідомлення;
- розробити можливість надсилати файли у форматі вкладень;
- розробити можливість переглядати усі чати;
- розробити механізм пошуку необхідного чату;
- розробити механізм фільтрації чатів;

- розробити можливість перегляду історії повідомлень з контактною особою;
- розробити можливість переприв'язувати чати до інших контактних осіб.

1.4.2 Розроблення нефункціональних вимог

До нефункціональних вимог можна віднести:

- створення загального чат-контролю в одному інтерфейсі на базі платформи CRM;
- коректна обробка вмісту повідомлень, надісланих контактними особами;
- обробка та візуальне позначення нових та непрочитаних повідомлень;
- можливість запустити інтерфейс у різних браузерях;
- програмне забезпечення повинно бути написано з використанням мов програмування C# та JavaScript, а також на базі системи керування базами даних Microsoft SQL Server.

1.4.3 Постановка комплексу завдань модулю

Необхідно інтегрувати ведення чатів у месенджерах в програмний функціонал роботи з лідами платформи CRM. Дане програмне забезпечення покликане підвищити продуктивність менеджерів, а саме автоматизувати та оптимізувати комунікацію з клієнтами в різних месенджерах.

Щоб досягнути поставленої мети, в розробленому програмному продукті має бути реалізовано такі особливості:

- об'єднати ведення листування із клієнтами на кількох платформах у єдиному інтерфейсі;
- створити можливість швидко фільтрувати та шукати необхідні чати;

					КПІ.ІП-8215.0454.30.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		19

- надати зручний для користувача інтерфейс для роботи з листуванням;
- обробити виняткові ситуації.

Для цього необхідно вивчити API чат-ботів обраних месенджерів для підключення автоматичного обміну даними з ними при спілкуванні менеджера з клієнтом. Також потрібно створити додаткове програмне забезпечення для відображення усієї історії листування та збереження обраних даних з прив'язкою до ліда та/або контактної особи.

Висновки до розділу

Внаслідок стрімкого розвитку технологій та впровадження їх на підприємства виникла потреба у веденні усіх процесів в одному місці. Саме цю проблему вирішують ERP-системи. Вони створені для зберігання та обробки даних про усі етапи виробництва, продажу, логістики, керування людськими ресурсами, обліку та бухгалтерії. Завдяки такому великому спектру функцій, які ERP-системи покликані реалізовувати, вони поділяються на функціональні зони, які можуть бути імплементовані разом або незалежно один від одного.

Однією із таких зон і є модуль CRM, що допомагає систематизувати та обробляти дані про потенційних клієнтів та продажі. Також важливою складовою є контроль етапів вирощування клієнта, адже ефективні продажі напряду залежать від своєчасності надання пропозицій від бізнесу. Саме для цих потреб існує безліч інструментів CRM-платформи, такі як: кваліфікація ліда, планування контактів з ним, стадійність, воронка продажів, угоди та документообіг. Так, у цій системі кожна угода проходить усі стадії бізнес-процесу, який дає змогу менеджеру аналізувати, що необхідно зробити на кожному етапі, аби угода стала успішною.

Задля ефективного використання людського ресурсу для менеджменту було розроблено чат-боти, що звільняють підприємство від необхідності винаймати більше працівників, аби забезпечити своєчасну консультацію

					КП.ІП-8215.0454.30.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		20

потенційних клієнтів. Чат-боти можна навчити самостійно давати відповіді на базові запитання, або ж вони можуть виступати посередниками між клієнтами та менеджерами, вчасно підключаючи останніх до розмови. Завдяки такій автоматизації бізнес отримує значні переваги, саме тому інтеграція чат-ботів у месенджерах та на веб-сайтах останнім часом здобула популярність.

На сьогоднішній день для розробників створено API для взаємодії з чат-ботами месенджерів з інших платформ, що дозволяє отримувати дані чату з різними людьми. Усі вони направлені на оптимізацію та зручність у використанні, проте в залежності від месенджера можливості функціоналу не завжди однакові.

Підсумовуючи опрацьовану інформацію було запропоновано інтегрувати ведення чатів у месенджерах в програмний функціонал роботи з лідами платформи CRM. Такий підхід до ведення контактів із потенційними клієнтами значно полегшує та оптимізує роботу менеджерів.

					КПІ.ІП-8215.045430.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		21

2 МОДЕЛЮВАННЯ ТА КОНСТРУЮВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

2.1 Моделювання та аналіз програмного забезпечення

Для розробки коректної архітектури програмного забезпечення застосунку важливим етапом є формалізація варіантів взаємодії з продуктом у вигляді діаграми прецедентів (use-case діграми). Серед основних акторів можна виділити:

- клієнта;
- менеджера;
- адміністратора.



Рисунок 2.1 – Діаграма прецедентів

Як можна побачити на рисунку 2.1 основними діями для клієнта є:

- розпочати розмову;
- надіслати повідомлення;
- надіслати вкладення.

Для менеджера визначені такі дії:

- надіслати повідомлення;
- надіслати вкладення;
- прив'язати розмову до ліда;
- прив'язати розмову до контрагента.

Важливо зазначити, що за політиками конфіденційності чат-бот не має можливості самостійно розпочинати розмову, оскільки так месенджери борються зі спамом. Саме тому у менеджера немає можливості першим розпочати розмову із клієнтом.

Адміністратор має такі основні дії:

- додати чат-бот;
- налаштувати чат-бот;
- видалити чат-бот.

Часто адміністратором та менеджером може бути один користувач, проте їхні ролі навмисно розділено налаштуваннями параметрів доступу.

2.2 Архітектура програмного забезпечення

Детальний опис публічних методів класів чат-ботів наведено в наступних таблицях:

Таблиця 2.1 — Публічні методи інтерфейсу IChatAggregatorMessageSender

Назва методу	Аргументи	Значення, що повертаються	Призначення методу
SendMessage	sendParams	SendMessageResult	Відправлення повідомлення через агрегатор чатів

Таблиця 2.2 — Публічні методи інтерфейсу IChatAggregatorApi

Назва методу	Аргументи	Значення, що повертаються	Призначення методу
SendMessage	sendMessageParams	SendMessageResult	Відправити повідомлення клієнту
GenerateEventData	jsonObject	ChatEventData	Створити дані події із рядка json-формату
GetClientAvatarUrl	clientId	string	Отримати посилання на аватар клієнта

Таблиця 2.3 — Публічні методи інтерфейсу IChatAggregatorConnection

Назва методу	Аргументи	Значення, що повертаються	Призначення методу
RemoveMasterWebhook	-	ChatApiRequestResult	Відключити основний webHook
MasterWebhookExists	-	WebhookSearchResult	Спробувати отримати webHook
CreateMasterWebhook	-	ChatApiRequestResult	Створити webHook
TestConnection	-	ChatApiRequestResult	Протестувати підключення webHook

Таблиця 2.4 — Публічні методи інтерфейсу IChatApiFactory

Назва методу	Аргументи	Значення, що повертаються	Призначення методу
GetApi	settings	IChatAggregatorApi	Отримати API

Таблиця 2.5 — Публічні методи інтерфейсу IChatApiSettingsProvider

Назва методу	Аргументи	Значення, що повертаються	Призначення методу
GetSettings	-	IChatApiSettings	Отримати налаштування API

Таблиця 2.6 — Публічні методи інтерфейсу IChatBotsRepository

Назва методу	Аргументи	Значення, що повертаються	Призначення методу
Update	chatBots, fieldsToUpdate	bool	Оновити/додати чат-боти
Retrieve	code	IChatBot	Отримати чат-бот по коду БД
Retrieve	botTextCode	IChatBot	Отримати чат-бот по зовнішньому коду
Add	chatBot	bool	Додати новий запис в БД

Таблиця 2.7 — Публічні методи інтерфейсу IClientBindingService

Назва методу	Аргументи	Значення, що повертаються	Призначення методу
GetBindedManagerCodes	clientExternalId	List<string>	Отримати список кодів менеджерів, пов'язаних з клієнтом чату
RebindLeadClients	leadCodeToUntie, contractorToBindCode, contactToBindCode	bool	Переприв'язати клієнта
GetBindedClients	bindableEntity	List<string>	Отримати список зовнішніх ключів клієнтів по зв'язаній сутності
GetBindedClients	managerCode	List<string>	Отримати список зовнішніх ключів клієнтів по коду відповідального менеджера

Продовження таблиці 2.7

Назва методу	Аргументи	Значення, що повертаються	Призначення методу
IsLinkedClient	clientCode	bool	Отримати ознаку чи закріплений клієнт за менеджером по коду в БД
IsLinkedClient	clientExtId	bool	Отримати ознаку чи закріплений клієнт за менеджером по зовнішньому коду
IsLinkedClient	client	bool	Отримати ознаку чи закріплений клієнт за менеджером
IsLinkedClient	clientCodes	Dictionary<int, bool>	Отримати словник із ознаками чи закріплені клієнти за менеджером

Таблиця 2.8 — Публічні методи інтерфейсу IClientsRepository

Назва методу	Аргументи	Значення, що повертаються	Призначення методу
Update	clients, fieldsToUpdate	bool	Оновити дані клієнтів
Retrieve	clientCode	IClient	Отримати клієнта по коду в БД
Retrieve	clientExternalId	IClient	Отримати клієнта по зовнішньому коду

Продовження таблиці 2.8

Назва методу	Аргументи	Значення, що повертаються	Призначення методу
Retrieve	clientCodes	List<ChatClient>	Отримати клієнта по кодах в БД
Retrieve	-	List<ChatClient>	Отримати усіх клієнтів
ClientExists	clientExternalId	bool	Отримати ознаку, чи існує клієнт з таким зовнішнім кодом
SetDialogReadSign	clientExtId, read	bool	Встановити діалогу ознаку «Прочитано»
RetrieveByLeadCode	leadCode	ChatClient	Отримати першого клієнта по коду ліда

Таблиця 2.9 — Публічні методи інтерфейсу IMessageRepository

Назва методу	Аргументи	Значення, що повертаються	Призначення методу
Update	messages, fieldsToUpdate	bool	Оновити/додати повідомлення
Add	message	int	Додати повідомлення в базу та отримати код
Retrieve	code, retrieveOriginals	IMessage	Отримати повідомлення по коду в БД з або без вкладень
Retrieve	messageExternalId, retrieveOriginals	IMessage	Отримати повідомлення по зовнішньому коду з або без вкладень

Продовження таблиці 2.9

Назва методу	Аргументи	Значення, що повертаються	Призначення методу
RetrieveList	clientExternalId, retrieveOriginals	List<IMessage>	Отримати усі повідомлення по зовнішньому коду клієнта з або без вкладень

Таблиця 2.10 — Публічні методи класу ChatAggregatorFrame

Назва методу	Аргументи	Значення, що повертаються	Призначення методу
AfterControlInit	-	-	Дія після ініціалізації контролю
AfterControlInitBefore ChatConstruction	chatCreationParams	-	Метод конфігурації параметрів контролю перед його створенням

Таблиця 2.11 — Публічні методи класу ClientBindingHelper

Назва методу	Аргументи	Значення, що повертаються	Призначення методу
AddClientPhoneDataToBinding	client	-	Додати номер телефону до прив'язаної контактної особи

2.3 Проектування бази даних

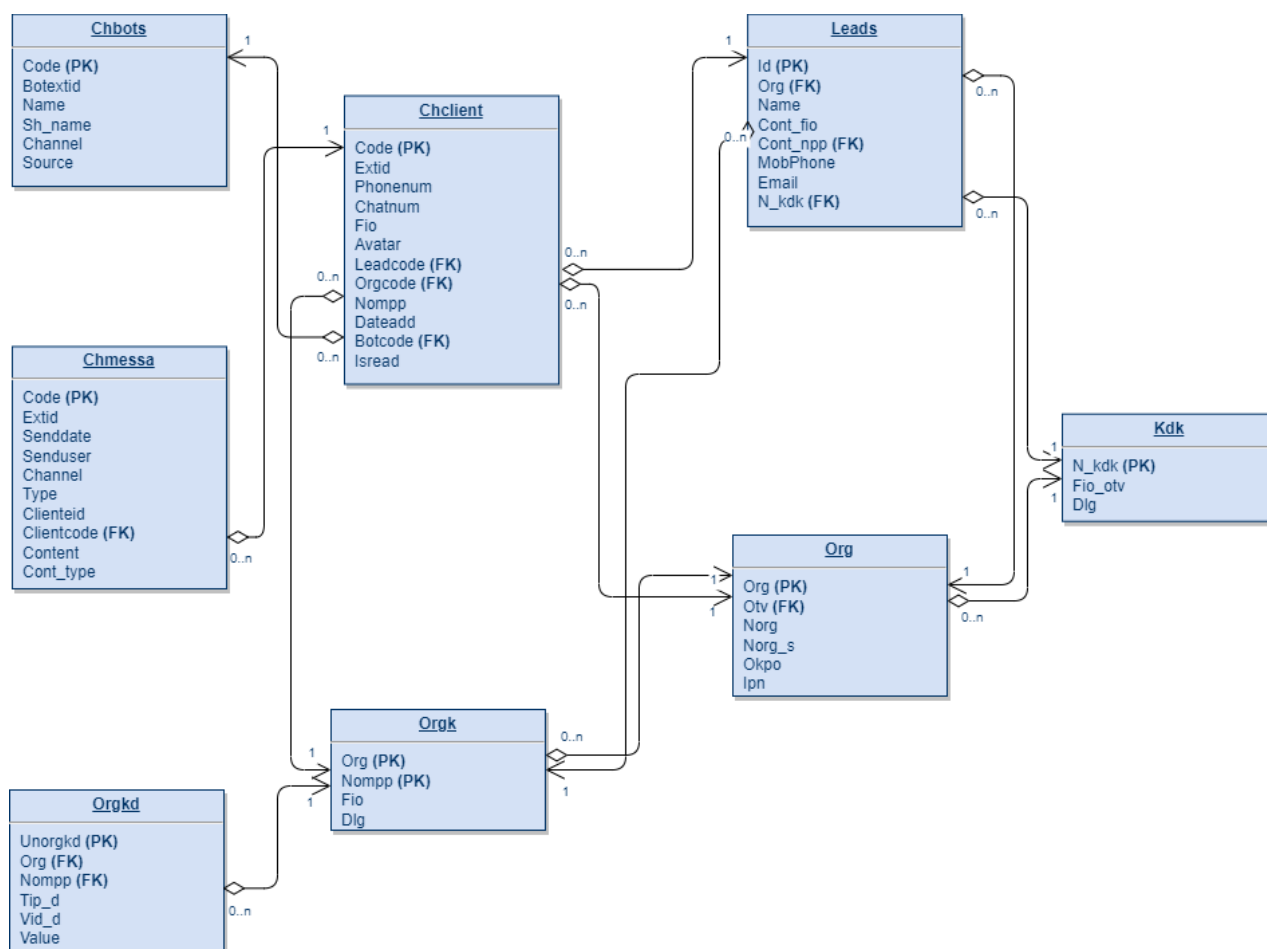


Рисунок 2.2 – Діаграма схеми бази даних

Схема бази даних, зображена на рисунку 2.2 показує основні таблиці, важливі для роботи із чат-ботами. Для виконання даного дипломного проекту було використано СКБД Microsoft SQL Server.

Таблиця 2.12 – Опис таблиць бази даних

Назва таблиці	Опис
Chbots	Таблиця із даними для налаштування чат-ботів
Chclient	Таблиця із даними клієнтів, що скористалися чат-ботами
Chmessa	Таблиця із повідомленнями, надісланими клієнтами
Leads	Ліди (потенційні клієнти)
Org	Контрагенти (кваліфіковані клієнти)
Orgk	Кваліфіковані контактні особи

Продовження таблиці 2.12

Назва таблиці	Опис
Orgkd	Контактні дані кваліфікованих контактних осіб
Kdk	Дані співробітників (менеджерів)

Варто зазначити, що структура таких таблиць як Leads, Org, Orgk, Orgkd та Kdk є спрощеною, адже зображені лише усі необхідні поля для повної роботи чат-ботів у розрізі модулю CRM, а не для усього модулю.

2.4 Аналіз безпеки даних

Для обміну даних через API застосовується протокол HTTPS, забезпечений з обох сторін. Таким чином розробники API попередньо впровадили шифрування даних, що надсилаються. Усі дані на боці розробленого програмного забезпечення зберігаються у хмарному сховищі і передаються також через протокол HTTPS, тому можна зробити висновок, що така схема обробки та збереження даних є цілком безпечною.

Висновки до розділу

В даному розділі було проаналізовано та сформульовано основні дії користувачів із програмним забезпеченням для роботи із чат-ботами, враховуючи можливі ролі користувачів. Виділені варіанти взаємодії було зображено у вигляді діаграми прецедентів.

Також було створено архітектуру застосунку, детально описано її модулі і основні методи розроблених класів. Дані описи було сформовано у вигляді таблиць.

Було розроблено схему бази даних і представлено у вигляді діаграми. Для роз'яснення її будови було описано створені таблиці.

Врешті, було проаналізовано безпеку даних, що передаються, обробляються та зберігаються у застосунку.

3 АНАЛІЗ ЯКОСТІ ТА ТЕСТУВАННЯ ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

3.1 Аналіз якості ПЗ

Для коректної оцінки якості програмного продукту необхідно провести аналіз його якості, що полягає у перевірці певного набору характеристик, які вимагаються від нього. Саме з цієї причини кожний продукт проходить процес тестування.

Тестування за рівнем автоматизації можна поділити на ручне (мануальне), напівавтоматизоване та автоматизоване. Для цього дипломного проєкту було обрано саме ручне тестування, адже покриття такого обсягу програмного забезпечення автотестами потребувало б забагато трудомісткості.

Для аналізу якості було виокремлено такі показники:

- переносність – наявність можливості використовувати програмне забезпечення у різних середовищах;
- функціональна повнота – наявність усіх функціональних засобів для роботи із даними;
- безвідмовність та стійкість до помилок – обробка та частота виникнення помилок при взаємодії із програмним продуктом;
- ефективність – характеристика, що позначає значну перевагу роботи з програмним забезпеченням над роботою без нього;
- зрозумілість та легкість використання – властивість, що позначає оптимальність функціоналу з точки зору складності навчання нового користувача.

3.2 Опис процесів тестування

Щоб провести тестування якості агрегатору чат-ботів необхідно поділити його на об'єкти оцінювання:

- користувацький інтерфейс;

- агрегатор запитів чат-ботів;
- адміністрування чат-ботів.

Охарактеризуємо компоненти тестування користувацького інтерфейсу:

- взаємодія із чатами менеджера;
- взаємодія із повідомленнями;
- взаємодія із контролем надсилання повідомлень;
- взаємодія із прив'язкою чату до профіля клієнта;
- наявність обробки помилок та виняткових ситуацій при роботі

користувача із інтерфейсом.

Такі ж компоненти опишемо для агрегатору чат-ботів:

- своєчасна та коректна обробка вхідних запитів від месенджерів;
- відправлення вихідних запитів із новими повідомленнями

користувачів;

- обробка виняткових ситуацій при роботі із вхідними та вихідними

запитами.

Врешті, вкажемо компоненти тестування адміністрування чат-ботів:

- можливість створити, коригувати та видалити чат-бота;
- взаємодія із списком клієнтів;
- взаємодія із повідомленнями та вкладеннями із чату.

При тестуванні варто враховувати, що воно буде проведене розробником, тому важливо не створювати лише позитивні сценарії тестування, які «правильні» з точки зору роботи функціоналу, а й негативні. Це можна зробити, використовуючи метод білої скриньки.

Таким чином сформуємо список функціональних тестів, які необхідно зробити, щоб проаналізувати якість розробки програмного забезпечення:

- тестування фрейму користувацького інтерфейсу: перевірка взаємодії із чатами;
- тестування фрейму користувацького інтерфейсу: перевірка взаємодії із повідомленнями;

- тестування фрейму користувацького інтерфейсу: перевірка взаємодії із контролем для створення нового повідомлення;
- тестування агрегатору: перевірка надсилання повідомлення;
- тестування агрегатору: перевірка отримання повідомлення;
- тестування агрегатору: перевірка обробки нового клієнта;
- тестування адміністрування: перевірка налаштування чат-боту;
- тестування адміністрування: перевірка можливості перегляду даних клієнтів.

3.3 Опис контрольного прикладу

Щоб перевірити коректність роботи створеного програмного забезпечення необхідно сформулювати контрольні приклади тестування.

Таблиця 3.1 – Сценарій перевірки алгоритму створення нового чату із клієнтом

Мета тесту	Перевірка алгоритму створення нового чату із клієнтом
Передумови	Створений чат-бот у месенджері Telegram та налаштовано доступ до даних, що передаються запитами через API
Схема виконання	Через месенджер почати роботу із ботом від імені клієнта, який до цього не взаємодіяв із ним
Очікуваний результат	У інтерфейсі налаштувань чат-ботів з'явився запис про клієнта. Було створено лід, що містить дані клієнта та прив'язаний до запису у інтерфейсі налаштувань
Дійсний результат	Створено запис про нового клієнта. З'явився новий лід, прив'язаний до клієнта

Таблиця 3.2 – Сценарій перевірки алгоритму надсилання текстового повідомлення з чат-контролю

Мета тесту	Перевірка алгоритму надсилання текстового повідомлення з чат-контролю
Передумови	Створений чат із клієнтом
Схема виконання	Зайти у чат із клієнтом, створити текстове повідомлення та надіслати його
Очікуваний результат	Текстове повідомлення буде записане у журнал. Через API буде надіслане повідомлення клієнту
Дійсний результат	У журналі з'явився запис із повідомленням. Клієнт отримав текстове повідомлення у месенджері

Таблиця 3.3 – Сценарій перевірки алгоритму надсилання повідомлення із вкладенням з чат-контролю

Мета тесту	Перевірка алгоритму надсилання повідомлення із вкладенням з чат-контролю
Передумови	Створений чат із клієнтом
Схема виконання	Зайти у чат із клієнтом, створити повідомлення із вкладенням та надіслати його
Очікуваний результат	Повідомлення із вкладенням буде записане у журнал. Через API буде надіслане вкладення клієнту
Дійсний результат	У журналі з'явився запис із повідомленням. Клієнт отримав вкладення у месенджері

Таблиця 3.4 – Сценарій перевірки алгоритму прив'язки відповідального менеджера до нового клієнта

Мета тесту	Перевірка алгоритму прив'язки відповідального менеджера до нового клієнта
Передумови	Новий клієнт розпочав листування
Схема виконання	Зайти у чат із клієнтом, написати повідомлення та надіслати його
Очікуваний результат	Відповідальним для ліда буде призначений менеджер, який перший відповів клієнту
Дійсний результат	Лід був прив'язаний до відповідального менеджера, що перший надіслав повідомлення клієнту

Таблиця 3.5 – Сценарій перевірки алгоритму обробки нового клієнта

Мета тесту	Перевірка алгоритму обробки нового клієнта
Передумови	Новий клієнт розпочав листування. Для ефективного маркетингу необхідно отримати номер телефону клієнта
Схема виконання	Надіслати повідомлення-привітання із запитом номеру телефону клієнта через кнопку, яка надсилає власний контакт співрозмовника
Очікуваний результат	Повідомлення буде записане у журнал. Через API буде надіслане повідомлення із кнопкою клієнту
Дійсний результат	У журналі з'явився запис із повідомленням. Клієнт отримав повідомлення із кнопкою у месенджері

Таблиця 3.6 – Сценарій перевірки алгоритму оновлення контактних даних клієнта

Мета тесту	Перевірка алгоритму оновлення контактних даних клієнта
Передумови	Надіслано повідомлення-привітання із запитом номеру телефону

Продовження таблиці 3.6

Схема виконання	Надіслати контакт від імені клієнта у месенджері
Очікуваний результат	Контактні дані клієнта записано у відповідну таблицю та лід
Дійсний результат	У таблиці клієнтів оновлено запис та додано номер телефону. У контактні дані ліда записано номер телефону

Кожен із сценаріїв, описаних у наведених вище таблицях, було послідовно протестовано.

Висновки до розділу

Для оцінки якості створеного програмного забезпечення необхідно детально протестувати створений функціонал. Важливо детально опрацювати усі сценарії та результати тестування, адже не повинно бути необроблених кейсів для застосунку, який видали у користування. Потенційні помилки можуть бути критичними для додатку, саме тому при тестуванні розробником потрібно якнайкраще змодельовати усі виняткові ситуації, що можуть виникнути у користувачів.

За результатами тестування усі сценарії були успішно відтворені та усі негативні тести були коректно оброблені.

4 ВПРОВАДЖЕННЯ ТА СУПРОВІД ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

4.1 Розгортання програмного забезпечення

Дане програмне забезпечення створене на базі платформи CRM, що уже розгорнуте у хмарному сховищі. Для додавання розробленого функціоналу до інструментів платформи було застосовано можливості Azure DevOps Server – програмного забезпечення для керування версіями.

Для розгортання цього функціоналу необхідно на власному сервері встановити ERP-застосунок із підключеним модулем CRM. Також потрібно розгорнути веб-застосунок, що оброблятиме запити месенджерів. Весь описаний функціонал уже буде включеним у цей додаток, тому потребуватиме лише конфігурації чат-ботів, описаної у наступному підрозділі.

4.2 Робота з програмним забезпеченням

Сторінка «Чати» необхідна, у випадку якщо у CRM використовуються чат-боти. На поточній сторінці є можливість читати та відповідати на повідомлення у чатах із власними та новими лідами.

На сторінці "Чати" відображаються чати та повідомлення. Закладка містить два блоки:

- список чатів – перелік листувань із лідами або кваліфікованими контактними особами;
- повідомлення – перелік повідомлень у листуванні.

Додавання нового чату відбувається автоматично, якщо контактна особа розпочала розмову із чат-ботом.

Якщо співрозмовник надав свій контакт, то номер телефону буде автоматично записано до контактних даних ліда або кваліфікованої контактної особи.

					КПІ.ІП-8215.0454.30.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		36

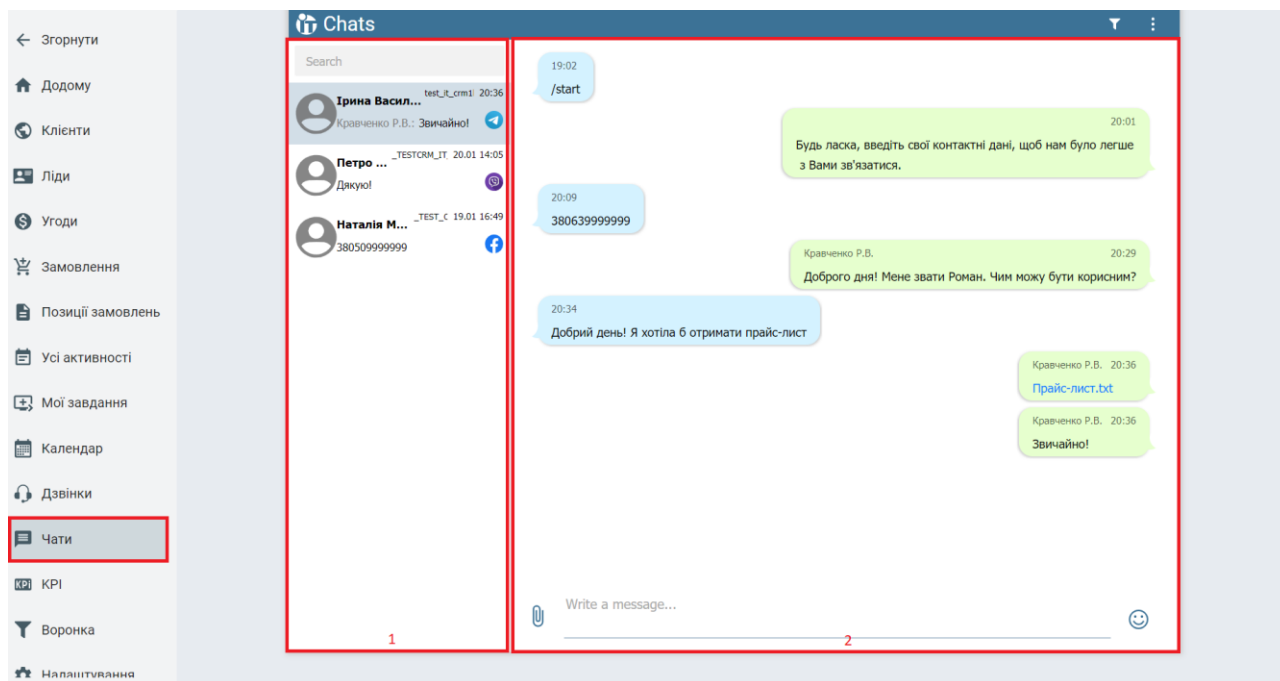


Рисунок 4.1 – Сторінка «Чати»

Якщо співрозмовник не надав свій контакт або по номеру телефону не було знайдено кваліфікованої контактної особи, то буде створено новий лід із його даними. Лід не матиме відповідального менеджера, та чат буде прив'язаний до ліда. Як тільки користувач напише повідомлення новому ліду, він буде автоматично призначеним як відповідальний ліда.

Якщо було знайдено кваліфіковану контактну особу по вказаному номеру телефону, то чат буде прив'язаний до неї.

У блоці «Чати» відображено перелік чатів із контактними особами. Сортювання відбувається по новіших повідомленнях. Чат має такі дані:

- позначення непрочитаного чату;
- ім'я контактної особи — ім'я, що задане в описі профілю у месенджері співрозмовника;
- останнє повідомлення у листуванні;
- іконка месенджера для інформації про джерело чату;
- позначення чату із контактною особою без відповідального менеджера;

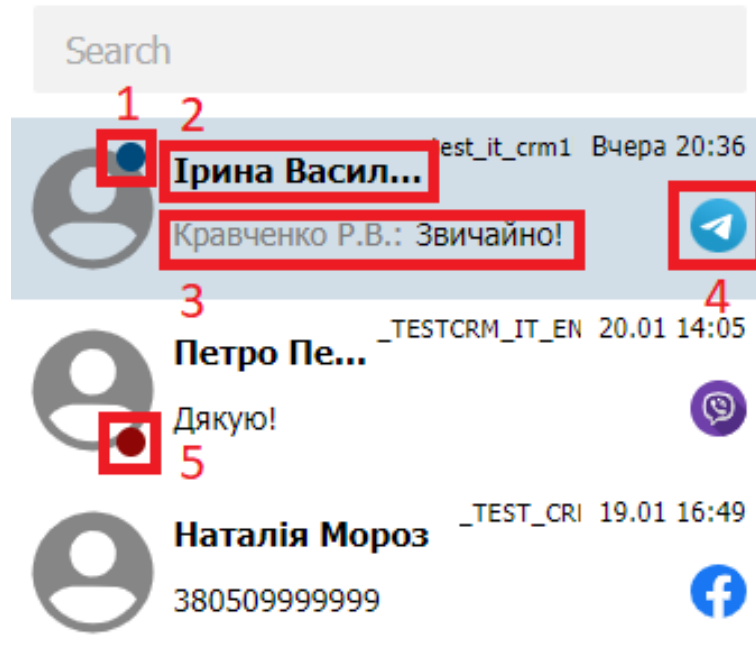


Рисунок 4.2 – Блок «Чати»

Для зручності пошуку потрібного чату є поле для пошуку. Щоб знайти чат із необхідною контактною особою, введіть у поле частину імені контакту (від 4 символів). У результаті буде відображено список чатів, для яких було знайдено збіги.

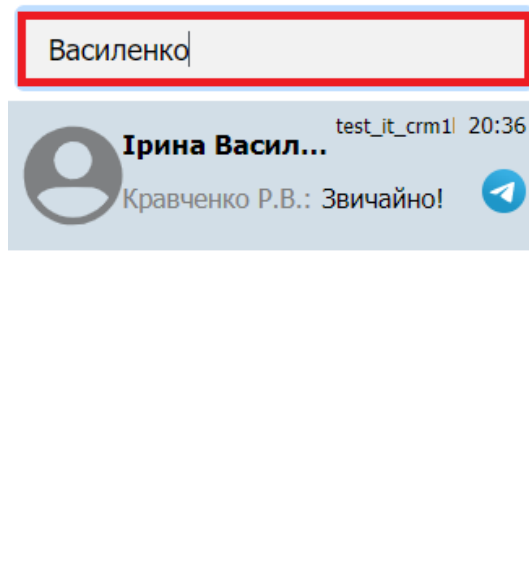


Рисунок 4.3 – Приклад використання поля пошуку чатів

Щоб відфільтрувати чати за заданими відборами, натисніть на кнопку «Фільтри» у верхньому лівому куті закладки. Буде відображено варіанти фільтрації чатів:

					КПІ.ІП-8215.0454.30.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		38

- всі чати;
 - мої чати — чати із контактними особами, для яких Ви є відповідальним менеджером;
 - неприв’язані чати — чати із контактними особами, для яких не заданий відповідальний менеджер;
 - непрочитані чати — чати із новими непрочитаними повідомленнями.
- Оберіть необхідний фільтр та натисніть на нього. Усі чати будуть відфільтровано згідно з обраним варіантом.

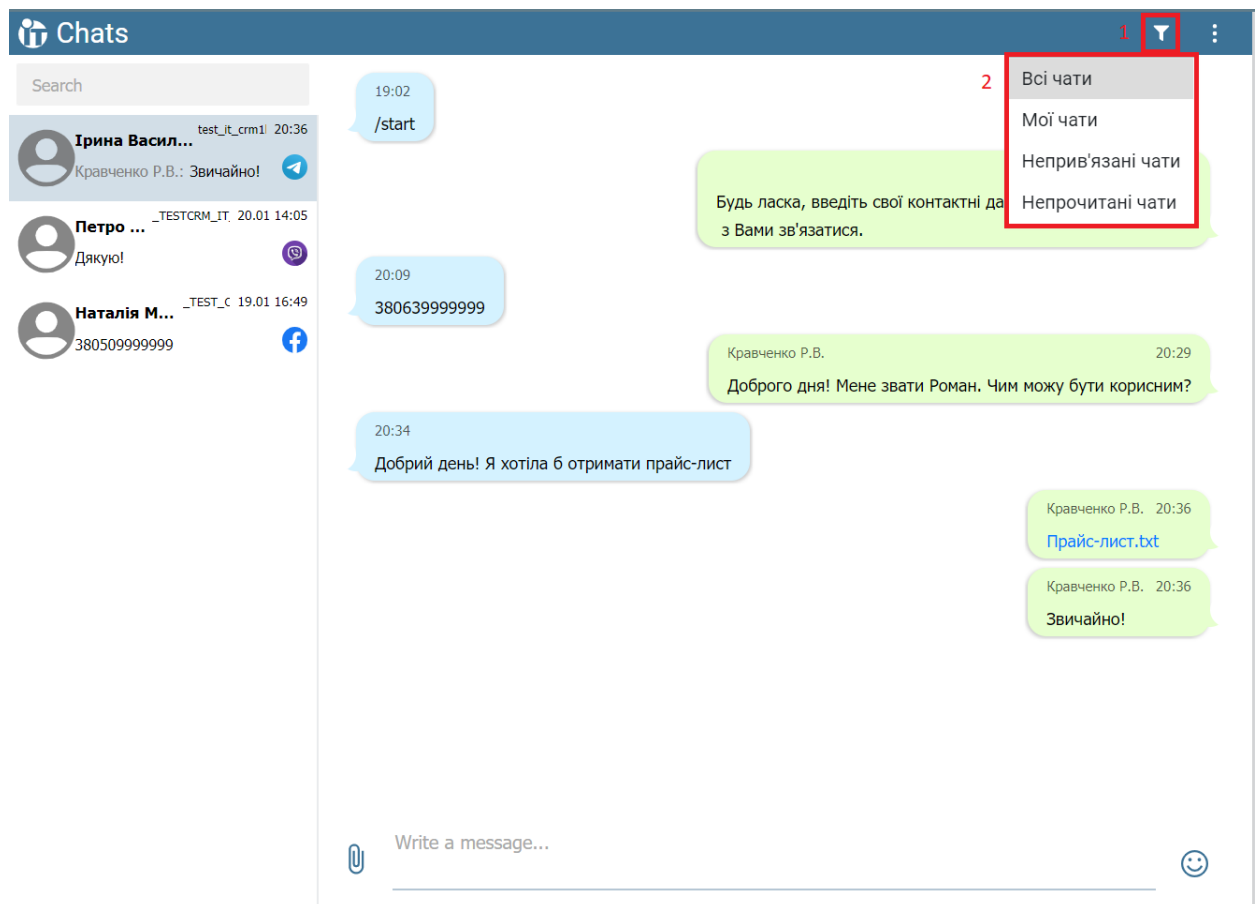


Рисунок 4.4 – Приклад фільтрування чатів

Щоб переприв’язати чат до іншої контактної особи, натисніть на виділену кнопку у верхньому лівому куті закладки та оберіть варіант «Переприв’язати чат».

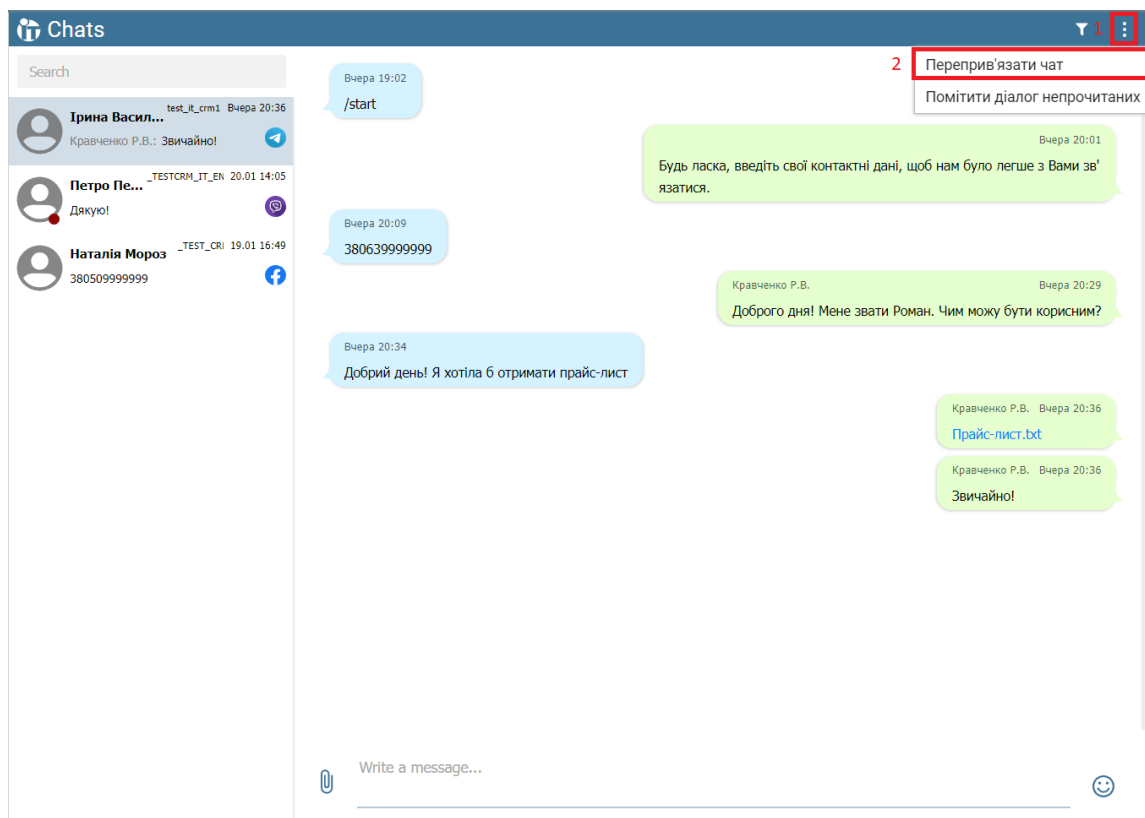


Рисунок 4.5 – Приклад переприв'язки чату до іншої контактної особи

Далі відкривається інтерфейс для вибору контактної особи. Оберіть необхідну особу, спозиціонуйтеся на ній та натисніть на кнопку «Вибрати».

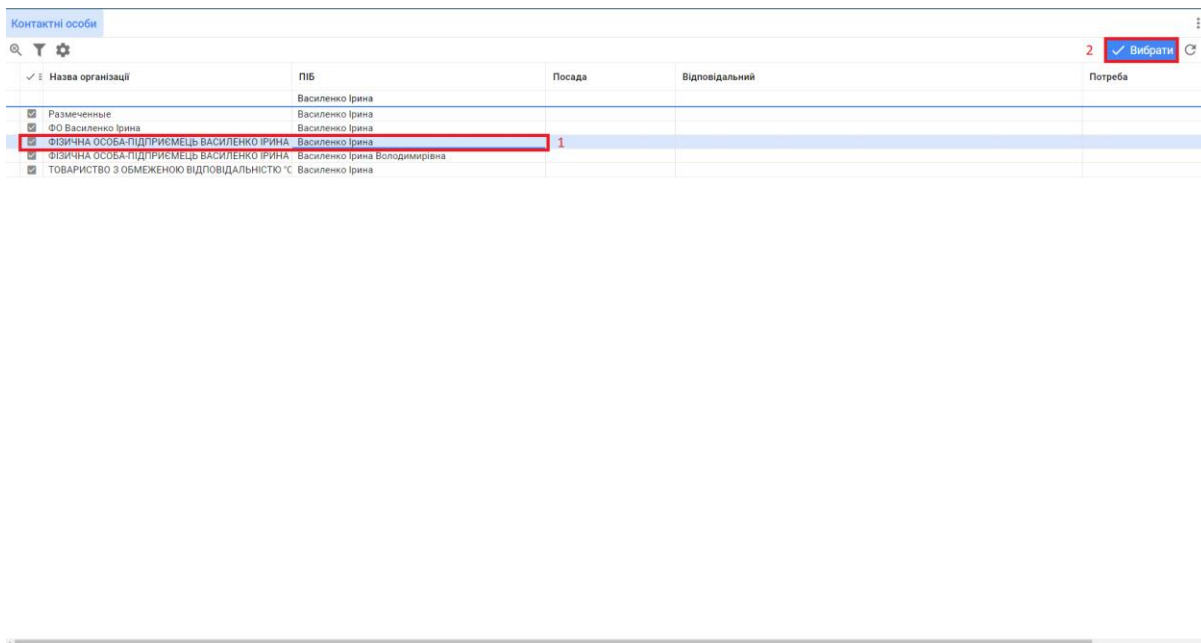


Рисунок 4.6 – Приклад вибору іншої контактної особи для прив'язки чату

Після цього чат буде автоматично переприв'язано до обраної контактної особи.

Щоб помітити чат як непрочитаний, натисніть на виділену кнопку у верхньому лівому куті закладки та оберіть варіант «Помітити діалог непрочитаним».

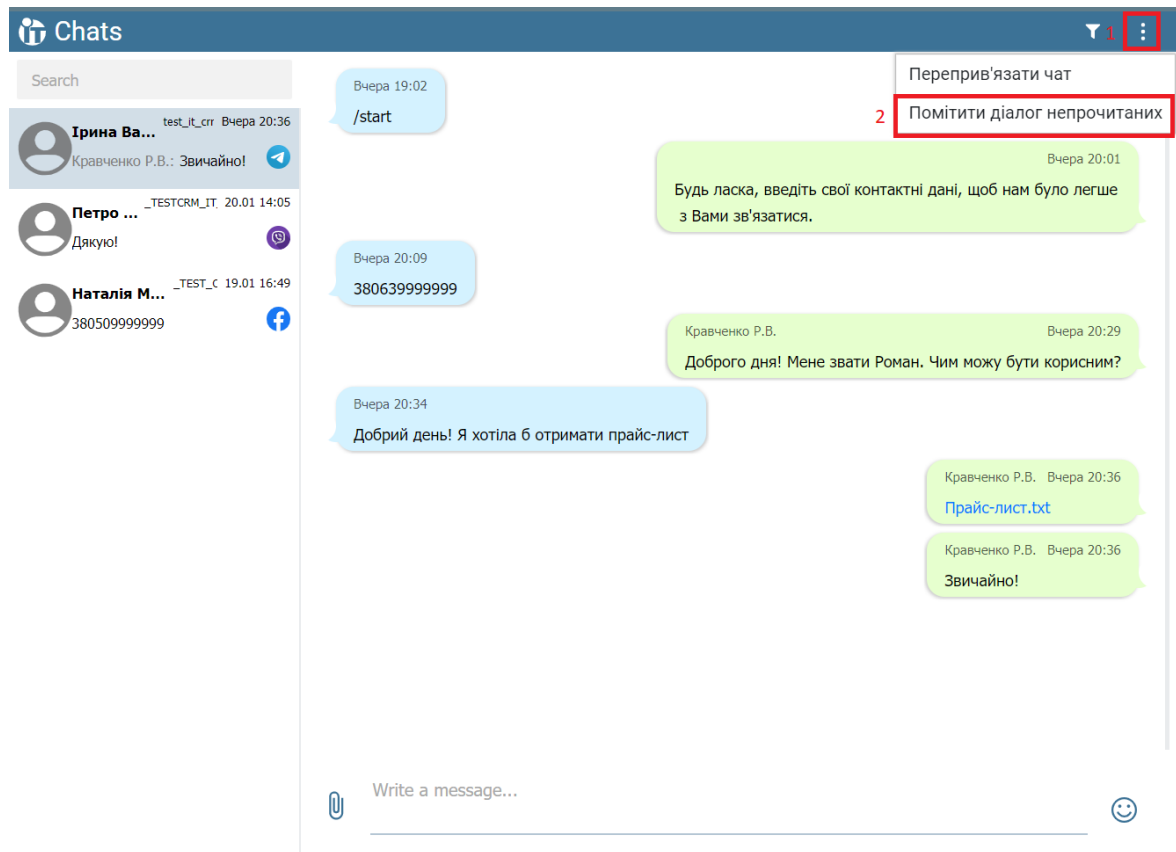


Рисунок 4.7 – Приклад позначення чату як непрочитаного

Після цього над фото в чаті з'явиться позначка непрочитаного чату.

У блоці «Повідомлення» відображено перелік повідомлень у обраному зліва чаті. Новіші повідомлення будуть відображені внизу. Повідомлення має такі дані:

- час надсилання повідомлення;
- автоматична відповідь бота на початок нової розмови;
- власні контактні дані, надіслані контактною особою;
- прізвище та ініціали відповідального менеджера;
- повідомлення, надіслане контактною особою;
- вкладення, надіслане менеджером.

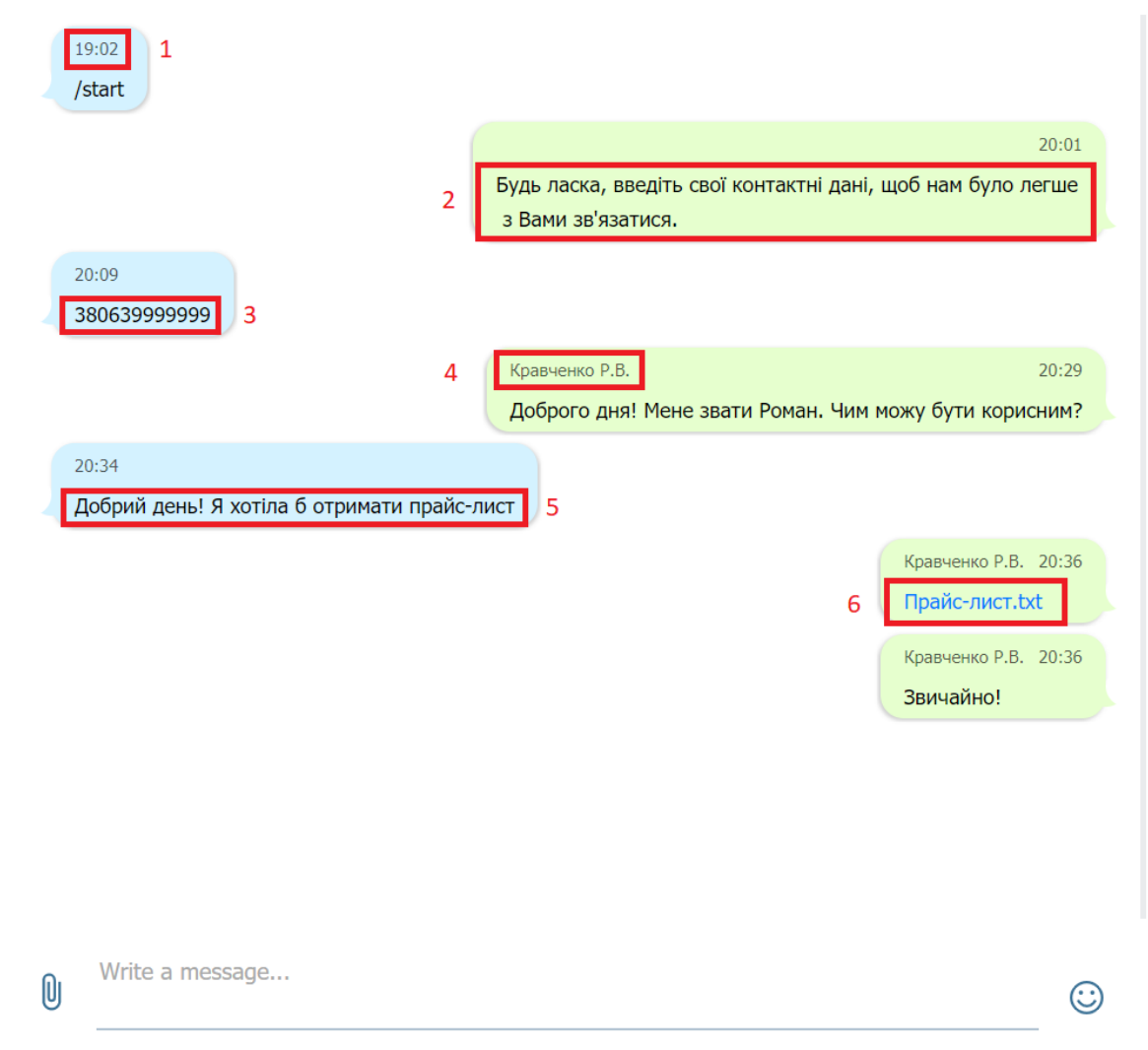


Рисунок 4.8 – Блок «Повідомлення»

Щоб надіслати повідомлення, введіть бажаний вміст у текстове поле, що знаходиться внизу блоку та натисніть кнопку «Відправити».



Рисунок 4.9 – Приклад надсилання повідомлення

Щоб додати емодзі, натисніть на кнопку, виділену нижче, та натисніть на обраний Вами емодзі.

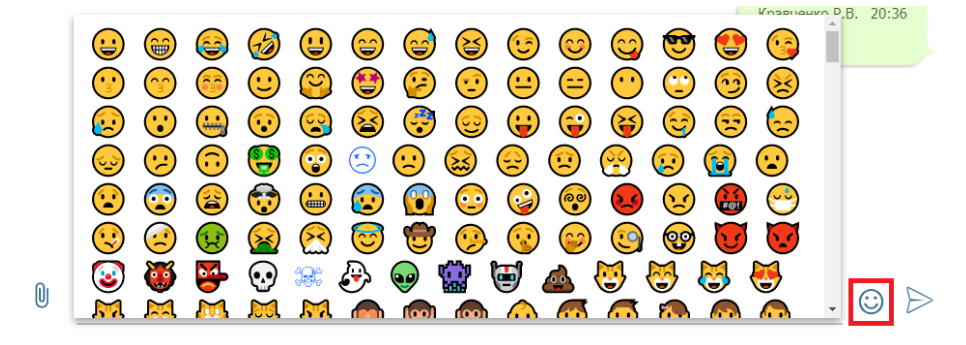


Рисунок 4.10 – Приклад додавання емодзі

Вибраний емодзі буде доданий у повідомлення.



Рисунок 4.11 – Приклад доданого в повідомлення емодзі

Щоб надіслати вкладення, натисніть на кнопку, виділену нижче.



Рисунок 4.12 – Приклад додавання вкладення

Далі з'явиться вікно, у якому потрібно спозиціонуватися на необхідний файл із Вашого комп'ютера та натиснути кнопку «Відкрити».

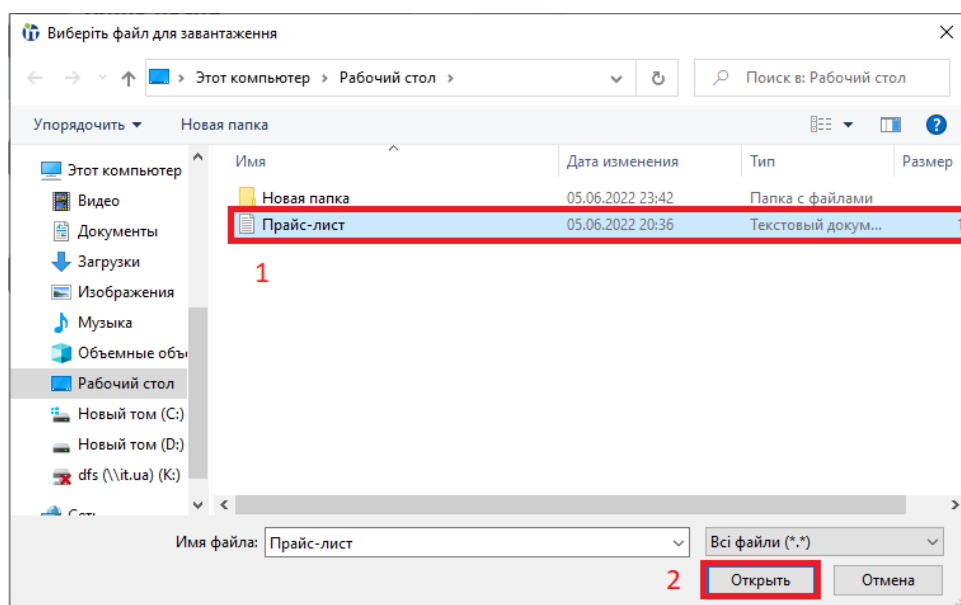


Рисунок 4.13 – Приклад вибору файлу вкладення

Обраний файл буде надісланий контактній особі. Якщо Вам необхідно отримати файл, надісланий Вами або співрозмовником, то наведіть курсор миші на назву файлу у повідомленні та натисніть на неї. Вкладення буде завантажене на Ваш комп'ютер.

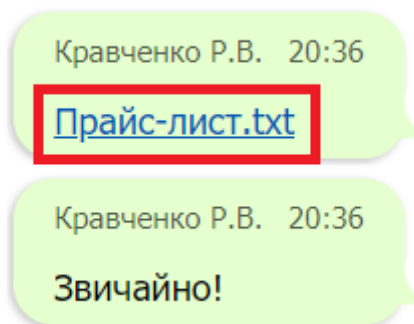


Рисунок 4.14 – Приклад завантаження файлу вкладення

Для налаштування агрегатора чат-ботів необхідно задати параметри. Щоб це було легко зробити, можна використовувати екран налаштування.

Налаштування інтеграції з агрегаторами чатів

☒ Використовувати агрегатори чатів
Виберіть сервіс ITChats

Тип потреби за замовчуванням DEMO. Реализация продукц
☒ Переприєязати клієнта після отримання тел.номери
☒ Закріплювати клієнта після першого вхідного повідомлення
☒ Відображати неприв'язаних клієнтів в діалогах
☐ Відправляти автовідповідь із запитом номера телефону
Текст автовідповідача

Текст кнопки

☐ Відправляти нотифікації
Тайм-аут нотифікації (сек)

0

Ліміт нотиф. останньому корист. (в сек)

0

Режим переходу до чату

ОК

Рисунок 4.15 – Налаштування інтеграції з агрегатором чатів

Опис налаштувань:

					КПІ.ІП-8215.0454.30.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		44

- використовувати агрегатори чатів – після активації даної ознаки в системі використовуватимуться чат-боти на відповідній сторінці в «Робочому просторі менеджера»;
- виберіть сервіс – в даному полі обирається сервіс, через який відбуватиметься інтеграція з чат-ботами;
- тип потреби за замовчуванням – тип потреби, з яким буде працювати чат-бот;
- переприв'язати клієнта після отримання тел. номеру – активація даної ознаки означає, що після отримання номеру телефону, в системі буде знайдено лід, який з ним пов'язаний, і виконається переприв'язка клієнта до відповідального за даний лід менеджера;
- закріплювати клієнта після першого вхідного повідомлення – після активації даної ознаки, клієнт закріплюється за менеджером, який перший йому відповів на повідомлення;
- відображати неприв'язаних клієнтів – активація даної ознаки дозволяє відображати в агрегаторі чатів клієнтів, які не прив'язані до менеджерів;
- відправляти автовідповідь із запитом номера телефону – після активації даної ознаки, з'явиться можливість автоматично відправляти текст автовідповідача на перше повідомлення клієнта;
- текст автовідповідача – поле для введення тексту повідомлення;
- текст кнопки – даний текст висвітлюватиметься на кнопці, яка слугує для того, щоб клієнт поділився номером телефону;
- відправляти нотифікації – після активації даної ознаки, в системі менеджеру приходитимуть сповіщення про новий чат;
- тайм-аут нотифікації (сек.) – в даному полі встановлюється кількість секунд, по закінченню яких, нотифікація буде прихована, якщо з нею не відбулась взаємодія. Мінімальне значення для відображення нотифікації – 5 секунд;

					КП.ІП-8215.0454.30.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		45

— ліміт нотиф. останньому корист. (в сек.) – при отриманні нового вхідного повідомлення аналізується останнє вихідне повідомлення від користувача. Якщо від моменту відправлення повідомлення пройшло менше заданого в даному полі часу – відбудеться надсилання нотифікації на даного користувача. Інакше – на всіх користувачів (у кого ввімкнені нотифікації);

— режим переходу до чату – в даному полі встановлюється режим переходу до чату: якщо під час переходу до чату з нотифікації, користувач знаходиться в «Робочому просторі менеджера», чат відкриється в даному інтерфейсі. В іншому випадку – чат відкриється відповідно до заданого режиму.

Для додавання налаштувань чат-боту потрібно перейти в інтерфейс «Чат-боти».

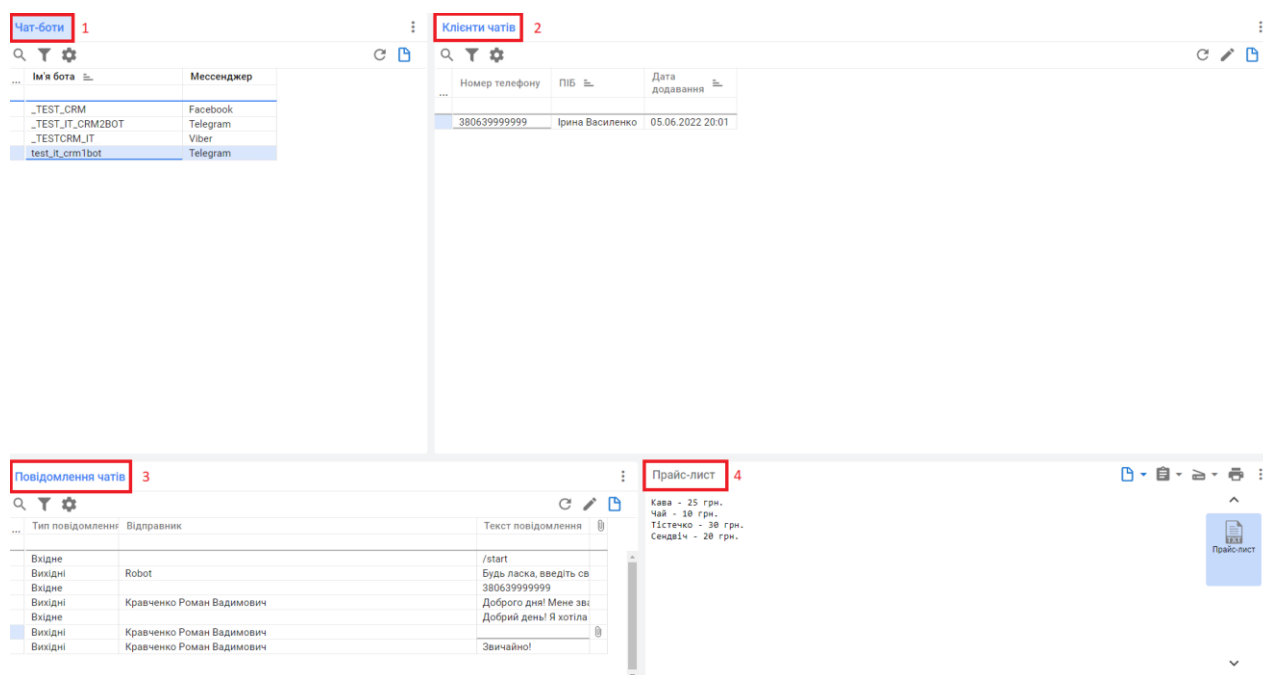


Рисунок 4.16 – Інтерфейс «Чат-боти»

Закладки інтерфейсу:

- чат-боти – перелік існуючих ботів;
- клієнти чатів – перелік прив'язаних клієнтів, що скористалися чат-ботом, вказаному зліва;
- повідомлення чатів – перелік повідомлень у чаті із клієнтом, зазначеним у попередньому пункті;

- вкладки – файл, вкладений у повідомлення.

Для роботи із параметрами чат-боту можна використовувати екран коригування.

Рисунок 4.17 – Екран коригування чат-бота

Опис параметрів:

- ім'я бота – унікальний код бота;
- ім'я (скорочено) – скорочене найменування бота. Використовується для відображення картки діалогу в чат-контролі та інтерактивній нотифікації нового повідомлення;
- месенджер – тип месенджера, у якому створено чат-бот;
- зовнішній код бота – код запису із налаштуваннями підключення до чат-боту;
- джерело нового ліда – код джерела ліда для подальшого визначення способу генерації.

Висновки до розділу

У даному розділі представлено детальне керівництво користувача, що дає змогу розпочати роботу із користувацьким інтерфейсом чат-ботів без

попереднього досвіду. Також таким чином можна проаналізувати способи взаємодії користувача із програмним забезпеченням.

Керівництво користувача містить у собі опис усіх дій та інструментів для зручного ведення продажів. Також у ньому представлено зображення із прикладами застосування описаного функціоналу.

Врешті, у керівництві було описано налаштування різних частин функціоналу чат-ботів, що можуть бути виконаними адміністратором. Створений програмний продукт дає змогу проводити персоніфікацію агрегатору під потреби різних користувачів.

					КПІ.ІП-8215.045430.02.81	Арк.
						48
Змін.	Арк.	№ докум.	Підп.	Дата		

ВИСНОВКИ

У даному дипломному проєкті проаналізовано актуальність використання чат-ботів для сучасного ведення бізнесу, види модулів платформи ERP та існуючі можливості API для роботи із чат-ботами різних месенджерів. Також описано процеси, що відбуваються для обробки потенційних клієнтів та формування комерційної угоди.

В результаті було спроектовано застосунок за допомогою мови програмування C# та СКБД Microsoft SQL Server. Таким чином було розроблено агрегатор для роботи із чат-ботами на платформі CRM.

Для виконання поставленої мети реалізовано такі вимоги:

- інтеграція чат-ботів із різних месенджерів;
- уніфікація даних, надісланих через API чат-ботів;
- розробка програмного забезпечення, що надає можливість оброблювати вхідні та вихідні запити;
- розробка користувацького чат-контролю для роботи із клієнтами;
- створення додаткових інструментів для зручної обробки чатів.

Розроблене програмне забезпечення пройшло різні сценарії та стадії тестування. Завдяки детальній перевірці створеного функціоналу було виявлено, опрацьовано та виправлено наявні помилки та виняткові ситуації.

Врешті, було сформовано проєктну документацію, схеми, табличні описи структури класів та сценаріїв тестування. Також було описано детальну інструкцію користувача, враховуючи налаштування агрегатору чат-ботів адміністратором.

					КП.ІП-8215.0454.30.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		49

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

- 1) What is ERP? [Електронний ресурс] – Режим доступу до ресурсу: <https://www.oracle.com/uk/erp/what-is-erp>.
- 2) Підприємство як об'єкт автоматизації [Електронний ресурс] – Режим доступу до ресурсу: https://elib.lntu.edu.ua/sites/default/files/elib_upload/%D0%90%D0%B2%D1%82%D0%BE%D0%BC%D0%B0%D1%82%D0%B8%D0%B7%D0%B0%D1%86%D1%96%D1%8F%20%D0%B1%D1%96%D0%B7%D0%BD%D0%B5%D1%81%20%D0%BF%D1%80%D0%BE%D1%86%D0%B5%D1%81%D1%96%D0%B2/2lec1.html.
- 3) Meena P. Customer Relationship Management Research from 2000 to 2020: An Academic Literature Review and Classification. *Vision-The Journal of Business Perspective*. 2021. 25 лютого.
- 4) Abubakr M. E. Tarig M. A. Haram A. Tarig A. A. Comparative study of customer relationship management (CRM) and electronic customer relationship management (E-CRM) *International Journal of Advanced and Applied Sciences*, 2021. 8 липня. С. 1-6.
- 5) Database Management Systems (DBMS) Comparison: MySQL, PostgreSQL, MSSQL Server, MongoDB, Elasticsearch, and others [Електронний ресурс] – Режим доступу до ресурсу: <https://www.altexsoft.com/blog/business/comparing-database-management-systems-mysql-postgresql-mssql-server-mongodb-elasticsearch-and-others/>.
- 6) Результати дослідження ринку CRM в Україні [Електронний ресурс] – Режим доступу до ресурсу: <https://auspex.ua/articles/biznes-sovety/rezultaty-doslidzhennya-rynku-crm-v-ukrayini/>.

					КП.ІП-8215.0454.30.02.81	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		50

ДОДАТКИ

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2022 р.

АГРЕГАТОР ЧАТ-БОТІВ ДЛЯ ОПТИМІЗАЦІЇ КОМУНІКАЦІЇ З КЛІЄНТАМИ З ПЛАТФОРМИ CRM

Опис програми

КП.ПІ-8215.045430.03.13

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олесь КОВТУНЕЦЬ

Нормоконтроль:

_____ Катерина ЛІЩУК

Виконавець:

_____ Ірина ЗОЗУЛЯ

Київ – 2022

ТЕКСТ ПРОГРАМНОГО КОДУ

Тексти програмного коду

Агрегатор чат-ботів для оптимізації комунікації з клієнтами з платформи CRM

(Найменування програми (документа))

CD-R

(Вид носія даних)

30 арк, 152 Кб

(Обсяг програми, арк., Кб)

Київ – 2022

					КП.ІП-8215.045430.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		2

IChatBot.cs

```
using ITnet2.Common.Tools;

namespace ITnet2.Server.BusinessLogic.CRM.PublicSurface.Chats
{
    /// <summary>
    /// Чат-бот
    /// </summary>
    public interface IChatBot
    {
        /// <summary>
        /// Код бота в БД
        /// </summary>
        int Code { get; set; }

        /// <summary>
        /// Внешний код бота
        /// </summary>
        string BotExtId { get; set; }

        /// <summary>
        /// Имя бота
        /// </summary>
        string Name { get; set; }

        /// <summary>
        /// Мессенджер в котором бот создан
        /// </summary>
        Channel Channel { get; set; }

        /// <summary>
        /// Код источника лида
        /// </summary>
        string SourceCode { get; set; }

        /// <summary>
        /// Chatbot type
        /// </summary>
        ChatBotType Type { get; }

        /// <summary>
        /// Use auto-reply for new chats
        /// </summary>
        bool AutoReplyToNewClient { get; }

        /// <summary>
        /// Auto-reply content
        /// </summary>
        string AutoReplyContent { get; }

        /// <summary>
        /// Auto-reply phone button label
        /// </summary>
        string AutoReplyPhoneButtonLabel { get; }

        /// <summary>
        /// Bot service type
        /// </summary>
        BotServiceType ServiceType { get; }

        /// <summary>
        /// Default needtype code for new leads
        /// </summary>
        string DefaultNeedtypeCode { get; }

        /// <summary>
        /// Try to rebind client after the phone is set
    }
}
```

					КП.ІІП-8215.045430.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата		3

```

    /// </summary>
    bool TryRebindClientAfterPhoneIsSet { get; }

    /// <summary>
    /// Auto bind manager to the client after the first sent message
    /// </summary>
    bool AutoBindingManager { get; }

    /// <summary>
    /// Show sign of not binded client
    /// </summary>
    bool ShowNotBindedClientSign { get; }
}

/// <summary>
/// Чат-бот
/// </summary>
public class ChatBot : IChatBot
{
    /// <summary>
    /// Код бота в БД
    /// </summary>
    [MapField("CODE")]
    public int Code { get; set; }

    /// <summary>
    /// Внешний код бота
    /// </summary>
    [MapField("BOTEXTID", CanMissing = MissingType.Sometimes)]
    public string BotExtId { get; set; }

    /// <summary>
    /// Имя бота
    /// </summary>
    [MapField("NAME", CanMissing = MissingType.Sometimes)]
    public string Name { get; set; }

    /// <summary>
    /// Мессенджер в котором бот создан
    /// </summary>
    [MapField("CHANNEL", CanMissing = MissingType.Sometimes)]
    public Channel Channel { get; set; }

    /// <summary>
    /// Код источника лида
    /// </summary>
    [MapField("SOURCE", CanMissing = MissingType.Sometimes)]
    public string SourceCode { get; set; }

    /// <summary>
    /// Chatbot type
    /// </summary>
    [MapField("BOTTYPE", CanMissing = MissingType.Sometimes)]
    public ChatBotType Type { get; set; }

    /// <summary>
    /// Use auto-reply for new chats
    /// </summary>
    [MapField("AUTOREPLY", CanMissing = MissingType.Sometimes)]
    private string _autoreplyToNewClientSign { get; set; }

    /// <summary>
    /// Use auto-reply for new chats
    /// </summary>
    [MapField(CanMissing = MissingType.Always)]
    public bool AutoReplyToNewClient
    {

```

					КП.ІП-8215.045430.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата		4

```

        get { return Text.Compare(_autoreplyToNewClientSign, "+"); }
        set { _autoreplyToNewClientSign = value ? "+" : string.Empty; }
    }

    /// <summary>
    /// Auto-reply content
    /// </summary>
    [MapField("REPLYCONT", CanMissing = MissingType.Sometimes)]
    public string AutoReplyContent { get; set; }

    /// <summary>
    /// Auto-reply phone button label
    /// </summary>
    [MapField("REPLYBTNLB", CanMissing = MissingType.Sometimes)]
    public string AutoReplyPhoneButtonLabel { get; set; }

    /// <summary>
    /// Bot service type
    /// </summary>
    [MapField("SERVICE", CanMissing = MissingType.Sometimes)]
    public BotServiceType ServiceType { get; set; }

    /// <summary>
    /// Default needtype code for new leads
    /// </summary>
    [MapField("NEEDTYPE", CanMissing = MissingType.Sometimes)]
    public string DefaultNeedtypeCode { get; set; }

    /// <summary>
    /// Try to rebind client after the phone is set
    /// </summary>
    [MapField("REBINDCLNT", CanMissing = MissingType.Sometimes)]
    private string _tryRebindClientAfterPhoneIsSetSign { get; set; }

    /// <summary>
    /// Try to rebind client after the phone is set
    /// </summary>
    [MapField(CanMissing = MissingType.Always)]
    public bool TryRebindClientAfterPhoneIsSet
    {
        get { return Text.Compare(_tryRebindClientAfterPhoneIsSetSign, "+"); }
        set { _tryRebindClientAfterPhoneIsSetSign = value ? "+" : string.Empty; }
    }

    }

    /// <summary>
    /// Auto bind manager to the client after the first sent message
    /// </summary>
    [MapField("AUTOBIND", CanMissing = MissingType.Sometimes)]
    private string _autoBindingManagerSign { get; set; }

    /// <summary>
    /// Auto bind manager to the client after the first sent message
    /// </summary>
    [MapField(CanMissing = MissingType.Always)]
    public bool AutoBindingManager
    {
        get { return Text.Compare(_autoBindingManagerSign, "+"); }
        set { _autoBindingManagerSign = value ? "+" : string.Empty; }
    }

    /// <summary>
    /// Show sign of not binded client
    /// </summary>
    [MapField("SHOWNOTBND", CanMissing = MissingType.Sometimes)]
    private string _showNotBindedClientSign { get; set; }

```

```

        /// <summary>
        /// Show sign of not binded client
        /// </summary>
        [MapField(CanMissing = MissingType.Always)]
        public bool ShowNotBindedClientSign
        {
            get { return Text.Compare(_showNotBindedClientSign, "+"); }
            set { _showNotBindedClientSign = value ? "+" : string.Empty; }
        }
    }

    /// <summary>
    /// Chatbot type
    /// </summary>
    public enum ChatBotType
    {
        /// <summary>
        /// Communication
        /// </summary>
        [MapValue("", CaptionResourceName = "Communication", IsDefaultValue = true)]
        Communication,

        /// <summary>
        /// Notification
        /// </summary>
        [MapValue("NF", CaptionResourceName = "Notification")]
        Notification
    }

    /// <summary>
    /// Bot service type
    /// </summary>
    public enum BotServiceType
    {
        /// <summary>
        /// CRM bots
        /// </summary>
        [MapValue("CRM", IsDefaultValue = true)]
        CrmBots,

        /// <summary>
        /// NovaChats
        /// </summary>
        [MapValue("NVC")]
        NovaChats,

        /// <summary>
        /// ITChats
        /// </summary>
        [MapValue("ITC")]
        ITChats
    }
}

```

IMessage.cs

```

using ITnet2.Common.Tools;
using System;

namespace ITnet2.Server.BusinessLogic.CRM.PublicSurface.Chats
{
    /// <summary>
    /// Сообщение чата
    /// </summary>
    public interface IMessage
    {
        /// <summary>

```

					КП.ІП-8215.045430.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата		6

```

/// Тип сообщения
/// </summary>
MessageType MessageType { get; set; }

/// <summary>
/// Код сообщения в БД
/// </summary>
int Code { get; set; }

/// <summary>
/// Текст сообщения
/// </summary>
string Text { get; set; }

/// <summary>
/// Идентификатор сообщения во внешнем сервисе
/// </summary>
string ExternalId { get; set; }

/// <summary>
/// Message number
/// </summary>
string MessageNumber { get; }

/// <summary>
/// Код клиента в БД
/// </summary>
int ClientCode { get; set; }

/// <summary>
/// Идентификатор клиента во внешнем сервисе
/// </summary>
string ExternalClientId { get; set; }

/// <summary>
/// Канал сообщения
/// </summary>
Channel Channel { get; set; }

/// <summary>
/// Признак отправки сообщения через CRM-систему
/// </summary>
bool SentByCRM { get; }

/// <summary>
/// Дата отправки
/// </summary>
DateTime SendDate { get; set; }

/// <summary>
/// Код менеджера
/// </summary>
string OperatorCode { get; set; }

/// <summary>
/// Тип контента сообщения
/// </summary>
MessageContentType ContentType { get; set; }

/// <summary>
/// Путь к файлу на сервере
/// </summary>
string ServerFilePath { get; set; }

/// <summary>
/// Ссылка на скачивание файла с вебсервиса
/// </summary>

```

					КП.ІП-8215.045430.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		7

```

        string FileUrl { get; set; }

        /// <summary>
        /// Признак того что файл - фото.
        /// </summary>
        bool IsPhoto { get; set; }

        /// <summary>
        /// Имя файла для отображения в контроле
        /// </summary>
        string FileName { get; set; }
    }

    /// <summary>
    /// 
    /// </summary>
    public enum MessageType
    {
        /// <summary>
        /// Входящее
        /// </summary>
        [MapValue("in", CaptionResourceName = "IncomingMessageType")]
        Incoming,

        /// <summary>
        /// Исходящее
        /// </summary>
        [MapValue("out", CaptionResourceName = "OutgoingMessageType")]
        Outgoing
    }

    /// <summary>
    /// Тип контента сообщения
    /// </summary>
    public enum MessageContentType
    {
        /// <summary>
        /// Текстовое сообщение
        /// </summary>
        [MapValue(" ", CaptionResourceName = "Text", IsDefaultValue = true)] Text,

        /// <summary>
        /// Файловое сообщение
        /// </summary>
        [MapValue("F", CaptionResourceName = "File")] File,
    }

    /// <summary>
    /// 
    /// </summary>
    public enum Channel
    {
        [MapValue("fb", Caption = "Facebook")]
        Facebook,
        [MapValue("wi", Caption = "Widget")]
        OnlineChat,
        [MapValue("tg", Caption = "Telegram")]
        Telegram,
        [MapValue("vb", Caption = "Viber")]
        Viber
    }

    public class ChatMessage : IMessage
    {
        [MapField("CODE")]
        public int Code { get; set; }
    }

```

```

    /// <summary>
    ///
    /// </summary>
    [MapField("TYPE", CanMissing = MissingType.Sometimes)]
    public MessageType MessageType { get; set; }

    [MapField("EXTID", CanMissing = MissingType.Sometimes)]
    public string ExternalId { get; set; }

    /// <summary>
    /// Message number
    /// </summary>
    [MapField("MESSAGENUM", CanMissing = MissingType.Sometimes)]
    public string MessageNumber { get; set; }

    [MapField("CLIENTCODE", CanMissing = MissingType.Sometimes)]
    public int ClientCode { get; set; }

    [MapField("CLIENTEID", CanMissing = MissingType.Sometimes)]
    public string ExternalClientId { get; set; }

    [MapField("CHANNEL", CanMissing = MissingType.Sometimes)]
    public Channel Channel { get; set; }

    [MapField(CanMissing = MissingType.Always)]
    public bool SentByCRM { get; set; }

    [MapField("SENDDATE", CanMissing = MissingType.Sometimes)]
    public DateTime SendDate { get; set; }

    [MapField("SENDUSER", CanMissing = MissingType.Sometimes)]
    public string OperatorCode { get; set; }

    [MapField("CONTENT", CanMissing = MissingType.Sometimes)]
    public string Text { get; set; }

    [MapField("CONT_TYPE", CanMissing = MissingType.Sometimes)]
    public MessageContentType ContentType { get; set; }

    [MapField(CanMissing = MissingType.Always)]
    public string ServerFilePath { get; set; }

    [MapField(CanMissing = MissingType.Always)]
    public string FileUrl { get; set; }

    [MapField(CanMissing = MissingType.Always)]
    public bool IsPhoto { get; set; }

    [MapField(CanMissing = MissingType.Always)]
    public string FileName { get; set; }

    public ChatMessage()
    {
        SentByCRM = false;
    }
}

/// <summary>
/// Кнопка в сообщении чат-бота
/// </summary>
public class MessageButton
{
    /// <summary>
    /// Тип кнопки
    /// </summary>
    public ButtonType ButtonType { get; set; }
    /// <summary>

```

					КП.ІП-8215.045430.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата		9

```

        /// Текст кнопки (желательно до 60 символов)
        /// </summary>
        public string Label { get; set; }
        /// <summary>
        /// Текст-ответ для соответствующего типа кнопок
        /// </summary>
        public string ResponseText { get; set; }
        /// <summary>
        /// Ссылка для перехода для соответствующего типа кнопок
        /// </summary>
        public string Link { get; set; }
    }

    /// <summary>
    /// Тип кнопки
    /// </summary>
    public enum ButtonType
    {
        /// <summary>
        /// Кнопка, нажатие которой возвращает текстовый ответ
        /// </summary>
        UserReply,
        /// <summary>
        /// Кнопка-запрос геолокации пользователя
        /// </summary>
        ShareLocation,
        /// <summary>
        /// Кнопка-запрос телефона пользователя
        /// </summary>
        SharePhone,
        /// <summary>
        /// Кнопка-запрос email
        /// </summary>
        ShareEmail,
        /// <summary>
        /// Кнопка-ссылка
        /// </summary>
        GoToLink
    }
}

```

IClient.cs

```

using ITnet2.Common.Tools;
using System;

/// <summary>
/// Интерфейсы для работы с чатами клиентов
/// </summary>
namespace ITnet2.Server.BusinessLogic.CRM.Bots.CommunicationBots
{
    /// <summary>
    /// Клиент - контактное лицо чата
    /// </summary>
    public interface IClient
    {
        /// <summary>
        /// Код сообщения в БД
        /// </summary>
        int Code { get; set; }

        /// <summary>
        /// Идентификатор клиента во внешнем сервисе
        /// </summary>
        string ExternalId { get; set; }

        /// <summary>

```

					КПИ.П-8215.045430.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата		10


```

каналов.
    /// Номер телефона, привязанный к клиенту. Доступен с начала чата не во всех
    /// Заполняется при ручном связывании клиента с контактными данными
    /// </summary>
    string PhoneNumber { get; set; }

    /// <summary>
    /// Имя клиента в мессенджере
    /// </summary>
    string Name { get; set; }

    /// <summary>
    /// Уникальный идентификатор пользователя в мессенджере. Может быть использован
    для генерации ссылок на прямое подключение
    /// </summary>
    string ChatNumber { get; set; }

    /// <summary>
    /// Url иконки аватара пользователя мессенджера
    /// </summary>
    string AvatarUrl { get; set; }

    /// <summary>
    /// Код привязанного лида
    /// </summary>
    int LeadCode { get; set; }

    /// <summary>
    /// Код контрагента, данного клиента
    /// </summary>
    int ContractorCode { get; set; }

    /// <summary>
    /// Код контактного лица
    /// </summary>
    int ContactCode { get; set; }

    /// <summary>
    /// Дата создания клиента
    /// </summary>
    DateTime CreationDate { get; set; }

    /// <summary>
    /// Код бота, через какого был создан клиент
    /// </summary>
    int BotCode { get; set; }

    /// <summary>
    /// Признак прочитанного диалога
    /// </summary>
    bool DialogIsRead { get; set; }
}

/// <summary>
/// Клиент чата с чат-ботами
/// </summary>
public class ChatClient : IClient
{
    /// <summary>
    /// Код сообщения в БД
    /// </summary>
    [MapField("CODE")]
    public int Code { get; set; }

    /// <summary>
    /// Идентификатор клиента во внешнем сервисе

```

каналах.

для генерации

```
/// </summary>
[MapField("EXTID", CanMissing = MissingType.Sometimes)]
public string ExternalId { get; set; }

/// <summary>
/// Номер телефона, привязанный к клиенту. Доступен с начала чата не во всех
/// Заполняется при ручном связывании клиента с контактными данными
/// </summary>
[MapField("PHONENUM", CanMissing = MissingType.Sometimes)]
public string PhoneNumber { get; set; }

/// <summary>
/// Имя клиента в мессенджере
/// </summary>
[MapField("FIO", CanMissing = MissingType.Sometimes)]
public string Name { get; set; }

/// <summary>
/// Url иконки аватара пользователя мессенджера
/// </summary>
[MapField("AVATAR", CanMissing = MissingType.Sometimes)]
public string AvatarUrl { get; set; }

/// <summary>
/// Код привязанного лида
/// </summary>
[MapField("LEADCODE", CanMissing = MissingType.Sometimes)]
public int LeadCode { get; set; }

/// <summary>
/// Код контрагента, данного клиента
/// </summary>
[MapField("ORGCODE", CanMissing = MissingType.Sometimes)]
public int ContractorCode { get; set; }

/// <summary>
/// Код контактного лица
/// </summary>
[MapField("NOMPP", CanMissing = MissingType.Sometimes)]
public int ContactCode { get; set; }

/// <summary>
/// Дата создания клиента
/// </summary>
[MapField("DATEADD", CanMissing = MissingType.Sometimes)]
public DateTime CreationDate { get; set; }

/// <summary>
/// Уникальный идентификатор пользователя в мессенджере. Может быть использован
/// для генерации ссылок на прямое подключение
/// </summary>
[MapField("CHATNUM", CanMissing = MissingType.Sometimes)]
public string ChatNumber { get; set; }

/// <summary>
/// Код бота, через какого был создан клиент
/// </summary>
[MapField("BOTCODE", CanMissing = MissingType.Sometimes)]
public int BotCode { get; set; }

/// <summary>
/// Признак прочитанного диалога
/// </summary>
[MapField(CanMissing = MissingType.Always)]
public bool DialogIsRead
{
```

```

        get => Text.Compare(_dialogIsRead, "+");
        set => _dialogIsRead = value ? "+" : "";
    }

    [MapField("ISREAD", CanMissing = MissingType.Sometimes)]
    private string _dialogIsRead { get; set; }
}

```

TelegramBotApi.cs

```

using ITnet2.Server.Session;
using ITnet2.Server.Session.Operations;
using Newtonsoft.Json;
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using System.Net;
using System.Net.Http;
using System.Text;
using System.Threading.Tasks;

namespace ITnet2.Server.BusinessLogic.CRM.Bots.Apis
{
    /// <summary>
    /// Telegram Bot API
    /// </summary>
    public class TelegramBotApi : IBotApi
    {
        /// <summary>
        /// Send a message to the subscriber
        /// </summary>
        /// <param name="message"></param>
        /// <param name="chatCode"></param>
        /// <param name="botApiKey">Telegram bot API key</param>
        /// <returns></returns>
        public IMessageSendingResult SendMessage(string message, string chatCode,
        string botApiKey)
        {
            if (string.IsNullOrEmpty(message) ||
                string.IsNullOrEmpty(chatCode) ||
                string.IsNullOrEmpty(botApiKey))
            {
                return new MessageSendingResult();
            }

            var parameters = new Dictionary<string, string>()
            {
                { "chat_id", chatCode },
                { "text", message },
                { "parse_mode", "HTML" }
            };

            var result = HttpPostRequest("sendMessage", botApiKey, parameters);
            return JsonConvert.DeserializeObject<MessageSendingResult>(result) ??
            new MessageSendingResult();
        }

        /// <summary>
        /// Send a file to the subscriber
        /// </summary>
        /// <param name="filePath"></param>
        /// <param name="chatCode"></param>
        /// <param name="botApiKey">Telegram bot API key</param>
        /// <returns></returns>
    }
}

```

```

        public IMessageSendingResult SendFile(string filePath, string chatCode, string
botApiKey)
        {
            if (string.IsNullOrEmpty(filePath) ||
                string.IsNullOrEmpty(chatCode) ||
                string.IsNullOrEmpty(botApiKey) ||
                !System.IO.File.Exists(filePath))
            {
                return new MessageSendingResult();
            }

            var name = Path.GetFileName(filePath);
            var parameters = new Dictionary<string, string>()
            {
                { "chat_id", chatCode }
            };

            var content = System.IO.File.OpenRead(filePath);

            var result = HttpPostRequestAsync("sendDocument", botApiKey, content,
name, parameters).Result;
            return JsonConvert.DeserializeObject<MessageSendingResult>(result) ??
new MessageSendingResult();
        }

        /// <summary>
        /// Get bot updates
        /// </summary>
        /// <param name="botApiKey">Telegram bot API key</param>
        /// <param name="lastOffset"></param>
        /// <returns></returns>
        public GetNewChatsResult GetBotUpdates(string botApiKey, string lastOffset =
null)
        {
            if (string.IsNullOrEmpty(botApiKey))
            {
                return new GetNewChatsResult();
            }

            var parameters = new Dictionary<string, string>();

            if (!string.IsNullOrEmpty(lastOffset))
            {
                parameters["offset"] = lastOffset;
            }

            var result =
JsonConvert.DeserializeObject<GetNewChatsResult>(HttpPostRequest("getUpdates", botApiKey,
parameters));

            return result ?? new GetNewChatsResult();
        }

        /// <summary>
        /// Send a phone number request to the subscriber
        /// </summary>
        /// <param name="botApiKey">Telegram bot API key</param>
        /// <param name="chatCode"></param>
        /// <param name="message"></param>
        /// <param name="buttonText"></param>
        /// <returns></returns>
        public IMessageSendingResult SendPhoneNumberRequest(string botApiKey, string
chatCode, string message, string buttonText)
        {
            if (string.IsNullOrEmpty(botApiKey) ||
                string.IsNullOrEmpty(chatCode) ||
                string.IsNullOrEmpty(message) ||
                string.IsNullOrEmpty(buttonText))
            {

```

```

        return new MessageSendingResult();
    }

    var parameters = new Dictionary<string, string>
    {
        { "chat_id", chatCode },
        { "text", message },
        { "reply_markup",
string.Concat(@"{"resize_keyboard":true,"one_time_keyboard":true,"keyboard":[[{"text":
"", buttonText, @"","request_contact":true}]]") }
    };
    var result = HttpPostRequest("sendMessage", botApiKey, parameters);
    return JsonConvert.DeserializeObject<MessageSendingResult>(result) ??
new MessageSendingResult();
}

/// <summary>
/// Set webhook for bot
/// </summary>
/// <param name="botApiKey">Viber bot API key</param>
/// <param name="botCode">Bot code</param>
/// <returns></returns>
public IApiResult SetWebhook(string botApiKey, int botCode)
{
    if (string.IsNullOrEmpty(botApiKey) ||
        botCode == 0)
    {
        return new ApiResult();
    }

    var parameters = new Dictionary<string, string>
    {
        { "url",
$"https://m.it.ua/ITDevelop/Deliveries/api/telegram/bots/{botCode}/event/" }
    };

    var result = HttpPostRequest("setWebhook", botApiKey, parameters);
    return JsonConvert.DeserializeObject<ApiResult>(result) ?? new
ApiResult();
}

/// <summary>
/// Delete webhook for bot
/// </summary>
/// <param name="botApiKey">Viber bot API key</param>
/// <returns></returns>
public IApiResult DeleteWebhook(string botApiKey)
{
    if (string.IsNullOrEmpty(botApiKey))
    {
        return new ApiResult();
    }

    var result = HttpPostRequest("deleteWebhook", botApiKey);
    return JsonConvert.DeserializeObject<ApiResult>(result) ?? new
ApiResult();
}

/// <summary>
/// Get server file path by file key
/// </summary>
/// <param name="botApiKey"></param>
/// <param name="fileKey"></param>
/// <param name="fileNameWithExtension"></param>
/// <returns></returns>
public string GetFilePath(string botApiKey, string fileKey, string
fileNameWithExtension = null)

```

```

{
    var url = Settings.Ust.GlCrm.TelegramApiLink;

    if (string.IsNullOrEmpty(url) ||
        string.IsNullOrEmpty(botApiKey) ||
        string.IsNullOrEmpty(fileKey))
    {
        return string.Empty;
    }

    var parameters = new Dictionary<string, string>
    {
        { "file_id", fileKey }
    };

    var fileGettingResult =
JsonConvert.DeserializeObject<FileGettingResult>(HttpPostRequest("getFile", botApiKey,
parameters))
        ?? new FileGettingResult();

    if (!fileGettingResult.Success ||
        fileGettingResult.File == null)
    {
        return string.Empty;
    }

    var filePath = fileGettingResult.File.FilePath;

    if (string.IsNullOrEmpty(filePath))
    {
        return string.Empty;
    }

    url += $"file/bot{botApiKey}/{filePath}";

    return Operation.DownloadHttpFile(url);
}

/// <summary>
/// Post a HTTP request to the Bot API
/// </summary>
/// <param name="method"></param>
/// <param name="botApiKey"></param>
/// <param name="parameters"></param>
/// <returns></returns>
public string HttpPostRequest(string method, string botApiKey,
Dictionary<string, string> parameters = null)
{
    if (string.IsNullOrEmpty(method) ||
        string.IsNullOrEmpty(botApiKey))
    {
        return string.Empty;
    }

    var url = Settings.Ust.GlCrm.TelegramApiLink;

    if (string.IsNullOrEmpty(url))
    {
        return string.Empty;
    }

    url += $"bot{botApiKey}/{method}";

    if (parameters != null && parameters.Count != 0)
    {
        url += "?";
    }
}

```

```

        url += string.Join("&", parameters.Select(item =>
string.Format("{0}={1}", item.Key, item.Value)));
    }

    HttpWebRequest request = (HttpWebRequest)WebRequest.Create(url);
    request.Method = "POST";
    request.ContentType = "application/x-www-form-urlencoded; charset=UTF-
8";

    HttpResponseMessage response;

    try
    {
        response = (HttpResponse)request.GetResponse();
    }
    catch
    {
        return string.Empty;
    }

    string jsonResponse;

    using (var dataStream = response.GetResponseStream())
    {
        using (var reader = new StreamReader(dataStream, Encoding.UTF8))
        {
            jsonResponse = reader.ReadToEnd();
        }
    }

    return jsonResponse;
}

/// <summary>
/// Post a HTTP request to the Bot API
/// </summary>
/// <param name="method"></param>
/// <param name="botApiKey"></param>
/// <param name="content"></param>
/// <param name="fileName"></param>
/// <param name="parameters"></param>
/// <returns></returns>
public async Task<string> HttpPostRequestAsync(string method, string botApiKey,
Stream content, string fileName, Dictionary<string, string> parameters = null)
{
    if (string.IsNullOrEmpty(method) ||
        string.IsNullOrEmpty(botApiKey) ||
        content == null ||
        string.IsNullOrEmpty(fileName))
    {
        return string.Empty;
    }

    var url = Settings.Ust.GlCrm.TelegramApiLink;

    if (string.IsNullOrEmpty(url))
    {
        return string.Empty;
    }

    url += $"bot{botApiKey}/{method}";

    var multipartContent = new
    MultipartFormDataContent($"{Guid.NewGuid()} {DateTime.UtcNow.Ticks}");

    foreach (var parameter in parameters)
    {

```

```

        multipartContent.Add(content: new
StringContent(parameter.Value.ToString()), name: parameter.Key);
    }

    var rawDisposition = $"form-data; name=""document"";
filename=""{fileName}""";
    var disposition = string.Join("",
Encoding.UTF8.GetBytes(rawDisposition).Select(c => Convert.ToChar(c)));

    // It will be dispose of after the request is made
#pragma warning disable CA2000
    var mediaPartContent = new StreamContent(content)
    {
        Headers =
        {
            {"Content-Type", "application/octet-stream"},
            {"Content-Disposition", disposition}
        }
    };
#pragma warning restore CA2000

    multipartContent.Add(mediaPartContent, "document", fileName);

#pragma warning disable CA2000
    var httpRequest = new HttpRequestMessage(HttpMethod.Post, url)
    {
        Content = multipartContent
    };
#pragma warning restore CA2000

    string jsonResponse;
    using (var client = new HttpClient())
    {
        using (var httpResponse = await client.SendAsync(httpRequest))
        {
            using (var dataStream = await
httpResponse.Content.ReadAsStreamAsync().ConfigureAwait(continueOnCapturedContext: false))
            {
                using (var reader = new StreamReader(dataStream,
Encoding.UTF8))
                {
                    jsonResponse = reader.ReadToEnd();
                }
            }
        }
    }

    return jsonResponse;
}

/// <summary>
/// API result
/// </summary>
public class ApiResult : IApiResult
{
    /// <summary>
    /// Sign of success (ok)
    /// </summary>
    [JsonProperty("ok")]
    public bool Success { get; set; }
}

/// <summary>
/// Message sending result
/// </summary>
public class MessageSendingResult : ApiResult, IMessageSendingResult
{

```



```

        /// <summary>
        /// Message (result)
        /// </summary>
        [JsonProperty("result")]
        private Message _message { get; set; }

        /// <summary>
        /// Message code (message_id)
        /// </summary>
        [JsonIgnore]
        public string MessageCode { get => _message?.MessageCode; }
    }

    /// <summary>
    /// New chats getting result
    /// </summary>
    public class GetNewChatsResult : ApiResult
    {
        /// <summary>
        /// Events (result)
        /// </summary>
        [JsonProperty("result")]
        public List<Event> Events { get; set; }
    }

    /// <summary>
    /// Event
    /// </summary>
    public class Event
    {
        /// <summary>
        /// Event code (update_id)
        /// </summary>
        [JsonProperty("update_id")]
        public string Code { get; set; }

        /// <summary>
        /// Message (message)
        /// </summary>
        [JsonProperty("message")]
        public Message Message { get; set; }
    }

    /// <summary>
    /// Message
    /// </summary>
    public class Message
    {
        /// <summary>
        /// Message code (message_id)
        /// </summary>
        [JsonProperty("message_id")]
        public string MessageCode { get; set; }

        /// <summary>
        /// Caption (caption)
        /// </summary>
        [JsonProperty("caption")]
        public string Caption { get; set; }

        /// <summary>
        /// Text (text)
        /// </summary>
        [JsonProperty("text")]
        public string Text { get; set; }

        /// <summary>

```

```

        /// Chat (chat)
        /// </summary>
        [JsonProperty("chat")]
        public Chat Chat { get; set; }

        /// <summary>
        /// Video (video)
        /// </summary>
        [JsonProperty("video")]
        public File Video { get; set; }

        /// <summary>
        /// Video (video)
        /// </summary>
        [JsonProperty("document")]
        public File Document { get; set; }

        /// <summary>
        /// Video (video)
        /// </summary>
        [JsonProperty("photo")]
        public List<File> Photos { get; set; }

        /// <summary>
        /// Contact (contact)
        /// </summary>
        [JsonProperty("contact")]
        public Contact Contact { get; set; }

        [JsonProperty("date")]
        private long _unixTimeStamp { get; set; }

        /// <summary>
        /// Date of message (date)
        /// </summary>
        public DateTime Date
        {
            get
            {
                var dtDateTime = new DateTime(1970, 1, 1, 0, 0, 0, 0,
DateTimeKind.Utc);
                return
dtDateTime.AddSeconds(_unixTimeStamp).ToLocalTime();
            }
        }

        /// <summary>
        /// File getting result
        /// </summary>
        private class FileGettingResult : ApiResult
        {
            /// <summary>
            /// File (result)
            /// </summary>
            [JsonProperty("result")]
            public File File { get; set; }
        }

        /// <summary>
        /// File
        /// </summary>
        public class File
        {
            /// <summary>
            /// File code (file_id)
            /// </summary>

```

					КП.ІІІ-8215.045430.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата		20

```

[JsonProperty("file_id")]
public string FileCode { get; set; }

/// <summary>
/// File path (file_path)
/// </summary>
[JsonProperty("file_path")]
public string FilePath { get; set; }

/// <summary>
/// File path (file_path)
/// </summary>
[JsonProperty("file_name")]
public string FileName { get; set; }

/// <summary>
/// File size (file_size)
/// </summary>
[JsonProperty("file_size")]
public int FileSize { get; set; }
}

/// <summary>
/// Chat
/// </summary>
public class Chat
{
    /// <summary>
    /// Chat code (id)
    /// </summary>
    [JsonProperty("id")]
    public string Code { get; set; }

    /// <summary>
    /// Telegram tag (username)
    /// </summary>
    [JsonProperty("username")]
    public string TelegramTag { get; set; }

    /// <summary>
    /// First name (first_name)
    /// </summary>
    [JsonProperty("first_name")]
    public string FirstName { get; set; }

    /// <summary>
    /// Last name (last_name)
    /// </summary>
    [JsonProperty("last_name")]
    public string LastName { get; set; }
}

/// <summary>
/// Contact
/// </summary>
public class Contact
{
    /// <summary>
    /// ChatCode (user_id)
    /// </summary>
    [JsonProperty("user_id")]
    public string ChatCode { get; set; }

    /// <summary>
    /// Phone number (phone_number)
    /// </summary>
    [JsonProperty("phone_number")]

```

```

        public string PhoneNumber { get; set; }

        /// <summary>
        /// First name (first_name)
        /// </summary>
        [JsonProperty("first_name")]
        public string FirstName { get; set; }

        /// <summary>
        /// Last name (last_name)
        /// </summary>
        [JsonProperty("last_name")]
        public string LastName { get; set; }
    }
}

```

ViberBotApi.cs

```

using ITnet2.Common.Tools;
using ITnet2.Server.BusinessLogic.Core.Originals;
using ITnet2.Server.Session;
using ITnet2.Server.Session.Operations;
using Newtonsoft.Json;
using Newtonsoft.Json.Converters;
using System.Collections.Generic;
using System.IO;
using System.Net;
using System.Runtime.Serialization;
using System.Text;

namespace ITnet2.Server.BusinessLogic.CRM.Bots.Apis
{
    /// <summary>
    /// Viber Bot API
    /// </summary>
    public class ViberBotApi : IBotApi
    {
        /// <summary>
        /// Send a message to the subscriber
        /// </summary>
        /// <param name="message"></param>
        /// <param name="chatCode"></param>
        /// <param name="botApiKey">Viber bot API key</param>
        /// <returns></returns>
        public IMessageSendingResult SendMessage(string message, string chatCode,
string botApiKey)
        {
            if (string.IsNullOrEmpty(message) ||
                string.IsNullOrEmpty(chatCode) ||
                string.IsNullOrEmpty(botApiKey))
            {
                return new MessageSendingResult();
            }

            var messageEntity = new Message()
            {
                Text = message,
                ChatCode = chatCode
            };

            var result = HttpPostRequest("send_message", botApiKey,
JsonConvert.SerializeObject(messageEntity));
            return JsonConvert.DeserializeObject<MessageSendingResult>(result) ??
new MessageSendingResult();
        }
    }
}

```

					КП.ІІІ-8215.045430.03.13	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		22

```

    /// <summary>
    /// Send a file to the subscriber
    /// </summary>
    /// <param name="filePath"></param>
    /// <param name="chatCode"></param>
    /// <param name="botApiKey">Telegram bot API key</param>
    /// <returns></returns>
    public IMessageSendingResult SendFile(string filePath, string chatCode, string
botApiKey)
    {
        if (string.IsNullOrEmpty(filePath) ||
            string.IsNullOrEmpty(chatCode) ||
            string.IsNullOrEmpty(botApiKey) ||
            !File.Exists(filePath))
        {
            return new MessageSendingResult();
        }

        var name = FileSystem.GetFileName(filePath,
FileSystem.GetFileNameType.WithExtensionWithoutVersion);

        var messageEntity = new Message()
        {
            ChatCode = chatCode,
            Type = "file",
            FileName = name,
            FileUrl =
$"https://m.it.ua/ITDevelop/ws/getfile.ashx?file={Settings.Ust.GlWeb.CopyFileToAllWebTemps(fil
ePath, true)}",/{Settings.Ust.GlWeb.Link}
            FileSize = (int)FileManager.GetClientFileInfo(filePath).Size
        };

        var result = HttpPostRequest("send_message", botApiKey,
JsonConvert.SerializeObject(messageEntity));
        return JsonConvert.DeserializeObject<MessageSendingResult>(result) ??
new MessageSendingResult();
    }

    /// <summary>
    /// Send a phone number request to the subscriber
    /// </summary>
    /// <param name="botApiKey">Viber bot API key</param>
    /// <param name="chatCode"></param>
    /// <param name="message"></param>
    /// <param name="buttonText"></param>
    /// <returns></returns>
    public IMessageSendingResult SendPhoneNumberRequest(string botApiKey, string
chatCode, string message, string buttonText)
    {
        if (string.IsNullOrEmpty(botApiKey) ||
            string.IsNullOrEmpty(chatCode) ||
            string.IsNullOrEmpty(message) ||
            string.IsNullOrEmpty(buttonText))
        {
            return new MessageSendingResult();
        }

        var buttons = new List<KeyboardButton>
        {
            new KeyboardButton()
            {
                ActionType = KeyboardActionType.SharePhone,
                Text = buttonText
            }
        };

        var messageEntity = new Message()

```

```

        {
            Text = message,
            ChatCode = chatCode,
            MinApiVersion = 7,
            Keyboard = new Keyboard()
            {
                Buttons = buttons
            }
        };

        var result = HttpPostRequest("send_message", botApiKey,
        JsonConvert.SerializeObject(messageEntity));
        return JsonConvert.DeserializeObject<MessageSendingResult>(result) ??
        new MessageSendingResult();
    }

    /// <summary>
    /// Set webhook for bot
    /// </summary>
    /// <param name="botApiKey">Viber bot API key</param>
    /// <param name="botCode"></param>
    /// <returns></returns>
    public IApiResult SetWebhook(string botApiKey, int botCode = 0)
    {
        if (string.IsNullOrEmpty(botApiKey))
        {
            return new ApiResult();
        }

        var webhook = new Webhook()
        {
            Url =
            "https://m.it.ua/ITDevelop/Deliveries/api/Viber/Post/" + $"{Settings.Ust.GlCrm.WebServiceLink}/
            api/Viber/Post/"
        };

        var result = HttpPostRequest("set_webhook", botApiKey,
        JsonConvert.SerializeObject(webhook));
        return JsonConvert.DeserializeObject<ApiResult>(result) ?? new
        ApiResult();
    }

    /// <summary>
    /// Delete webhook for bot
    /// </summary>
    /// <param name="botApiKey">Viber bot API key</param>
    /// <returns></returns>
    public IApiResult DeleteWebhook(string botApiKey)
    {
        if (string.IsNullOrEmpty(botApiKey))
        {
            return new ApiResult();
        }

        var webhook = new Webhook()
        {
            Url = string.Empty
        };

        var result = HttpPostRequest("set_webhook", botApiKey,
        JsonConvert.SerializeObject(webhook));
        return JsonConvert.DeserializeObject<ApiResult>(result) ?? new
        ApiResult();
    }

    /// <summary>
    /// Get server file path by file key

```

```

    /// </summary>
    /// <param name="botApiKey">Url for media</param>
    /// <param name="fileKey"></param>
    /// <param name="fileNameWithExtension"></param>
    /// <returns></returns>
    public string GetFilePath(string botApiKey, string fileKey, string
fileNameWithExtension)
    {
        if (string.IsNullOrEmpty(botApiKey) ||
            string.IsNullOrEmpty(fileKey))
        {
            return string.Empty;
        }

        var request = (HttpWebRequest)WebRequest.Create(fileKey);
        request.Method = "GET";
        request.ContentType = "application/json";

        HttpWebResponse response;

        try
        {
            response = (HttpWebResponse)request.GetResponse();
        }
        catch
        {
            return string.Empty;
        }

        var bufferSize = (int)response.ContentLength;
        var buffer = new byte[bufferSize];
        var bytesRead = 0;

        var path =
Originals.GetUniqueServerFileName(Settings.Environment.TempDir, fileNameWithExtension);
        FileStream fileStream = File.Create(path);

        using (var dataStream = response.GetResponseStream())
        {
            while ((bytesRead = dataStream.Read(buffer, 0, bufferSize)) != 0)
            {
                fileStream.Write(buffer, 0, bytesRead);
            }
        }

        return path;
    }

    /// <summary>
    /// Post a HTTP request to the Bot API
    /// </summary>
    /// <param name="method"></param>
    /// <param name="botApiKey"></param>
    /// <param name="jsonString"></param>
    /// <returns></returns>
    public string HttpPostRequest(string method, string botApiKey, string
jsonString)
    {
        if (string.IsNullOrEmpty(method) ||
            string.IsNullOrEmpty(botApiKey))
        {
            return string.Empty;
        }

        var url = Settings.Ust.GlCrm.ViberApiLink;

        if (string.IsNullOrEmpty(url))

```

```

        {
            return string.Empty;
        }

        url += "${method}";
        var request = (HttpWebRequest)WebRequest.Create(url);
        request.Method = "POST";
        request.ContentType = "application/json";
        request.Headers.Add("X-Viber-Auth-Token", botApiKey);

        if (!string.IsNullOrEmpty(jsonString))
        {
            using (var streamWriter = new
StreamWriter(request.GetRequestStream()))
            {
                streamWriter.Write(jsonString);
            }
        }

        HttpWebResponse response;

        try
        {
            response = (HttpWebResponse)request.GetResponse();
        }
        catch
        {
            return string.Empty;
        }

        string jsonResponse;
        using (var dataStream = response.GetResponseStream())
        {
            using (var reader = new StreamReader(dataStream, Encoding.UTF8))
            {
                jsonResponse = reader.ReadToEnd();
            }
        }

        return jsonResponse;
    }

    /// <summary>
    /// API result
    /// </summary>
    public class ApiResult : IApiResult
    {
        /// <summary>
        /// Status (status)
        /// </summary>
        [JsonProperty("status")]
        private int _status { get; set; } = -1;

        /// <summary>
        /// Sign of success
        /// </summary>
        [JsonIgnore]
        public bool Success { get => _status == 0; }
    }

    /// <summary>
    /// Message sending result
    /// </summary>
    public class MessageSendingResult : ApiResult, IMessageSendingResult
    {
        /// <summary>
        /// Message code (message_token)

```



```

        /// </summary>
        [JsonProperty("message_token")]
        public string MessageCode { get; set; }
    }

    /// <summary>
    /// Event
    /// </summary>
    private class Event
    {
        /// <summary>
        /// Event code (update_id)
        /// </summary>
        [JsonProperty("update_id")]
        public string Code { get; set; }

        /// <summary>
        /// Message (message)
        /// </summary>
        [JsonProperty("message")]
        public Message Message { get; set; }
    }

    /// <summary>
    /// Message
    /// </summary>
    private class Message
    {
        /// <summary>
        /// Chat Code (receiver)
        /// </summary>
        [JsonProperty("receiver")]
        public string ChatCode { get; set; }

        /// <summary>
        /// Text (text)
        /// </summary>
        [JsonProperty("text")]
        public string Text { get; set; }

        /// <summary>
        /// Minimal API version (min_api_version). For text messages - 1, for
messages with buttons - 7
        /// </summary>
        [JsonProperty("min_api_version")]
        public int MinApiVersion { get; set; } = 1;

        /// <summary>
        /// Keyboard (keyboard)
        /// </summary>
        [JsonProperty("keyboard")]
        public Keyboard Keyboard { get; set; }

        /// <summary>
        /// Message type (type)
        /// </summary>
        [JsonProperty("type")]
        public string Type { get; set; } = "text";

        /// <summary>
        /// File url (media)
        /// </summary>
        [JsonProperty("media")]
        public string FileUrl { get; set; }

        /// <summary>
        /// File name (file_name)

```

```

        /// </summary>
        [JsonProperty("file_name")]
        public string FileName { get; set; }

        /// <summary>
        /// File size (size)
        /// </summary>
        [JsonProperty("size")]
        public int FileSize { get; set; }
    }

    /// <summary>
    /// Keyboard object
    /// </summary>
    private class Keyboard
    {
        /// <summary>
        /// Array containing all keyboard buttons by order
        /// </summary>
        [JsonProperty("Buttons")]
        public List<KeyboardButton> Buttons { get; set; }

        /// <summary>
        /// Short keyboards will be displayed with the minimal possible height.
        /// Maximal height will be native keyboard height
        /// </summary>
        [JsonProperty("DefaultHeight")]
        private const bool _defaultHeight = false;

        /// <summary>
        /// Const value.
        /// </summary>
        [JsonProperty("Type")]
        private const string _type = "keyboard";
    }

    /// <summary>
    /// Keyboard button.
    /// </summary>
    private class KeyboardButton
    {
        /// <summary>
        /// Type of action pressing the button will perform
        /// </summary>
        /// <remarks>Default value: <see
        cref="KeyboardActionType.Reply"/>.</remarks>
        [JsonProperty("ActionType")]
        public KeyboardActionType? ActionType { get; set; }

        /// <summary>
        /// Text for 'reply' and 'none'. ActionType or URL for 'open-url'
        /// </summary>
        /// <remarks>For ActionType 'reply' - text. For ActionType 'open-url' -
        Valid URL.</remarks>
        [JsonProperty("ActionBody")]
        public string ActionBody { get; set; }

        /// <summary>
        /// Text to be displayed on the button. Can contain some HTML tags
        /// </summary>
        /// <remarks>
        /// Free text. Valid and allowed HTML tags Max 250 characters.
        /// If the text is too long to display on the button it will be cropped
        and ended with "...
        /// </remarks>
        [JsonProperty("Text")]
        public string Text { get; set; }
    }

```

```

    /// <summary>
    /// Button width in columns
    /// </summary>
    /// <remarks>Possible values: 1-6. Default value: 6.</remarks>
    [JsonProperty("Columns")]
    public int? Columns { get; set; }

    /// <summary>
    /// Button height in rows.
    /// </summary>
    /// <remarks>Possible values: 1-2. Default value: 1.</remarks>
    [JsonProperty("Rows")]
    public int? Rows { get; set; }

    /// <summary>
    /// Background color of button (valid color HEX value).
    /// </summary>
    [JsonProperty("BgColor")]
    public string BackgroundColor { get; set; }

    /// <summary>
    /// Determine whether the user action is presented in the conversation.
    /// </summary>
    [JsonProperty("Silent")]
    public bool? Silent { get; set; }

    /// <summary>
    /// URL for background media content (picture or gif). Will be placed
with aspect to fill logic.
    /// </summary>
    [JsonProperty("BgMedia")]
    public string BackgroundMedia { get; set; }

    /// <summary>
    /// URL of image to place on top of background (if any).
    /// Can be a partially transparent image that will allow showing some of
the background.
    /// Will be placed with aspect to fill logic.
    /// </summary>
    /// <remarks>JPEG and PNG files are supported. Max size: 500
kb.</remarks>
    [JsonProperty("Image")]
    public string Image { get; set; }
}

/// <summary>
/// Webhook
/// </summary>
private class Webhook
{
    /// <summary>
    /// Url (url)
    /// </summary>
    [JsonProperty("url")]
    public string Url { get; set; }

    /// <summary>
    /// Sign of sending name (send_name)
    /// </summary>
    [JsonProperty("send_name")]
    private const bool _sendName = true;

    /// <summary>
    /// Sign of sending photo (send_photo)
    /// </summary>
    [JsonProperty("send_photo")]

```

```

private const bool _sendPhoto = false;

/// <summary>
/// Event types (event_types)
/// </summary>
[JsonProperty("event_types")]
private static readonly List<string> _eventType = new List<string>
{
    "subscribed",
    "conversation_started"
};
}

/// <summary>
/// Keyboard action type.
/// </summary>
[JsonConverter(typeof(StringEnumConverter))]
private enum KeyboardActionType
{
    /// <summary>
    /// The value of ActionBody is sent as a text message to account
    /// </summary>
    [EnumMember(Value = "reply")]
    Reply = 1,

    /// <summary>
    /// The value of ActionBody is opened in the browser
    /// </summary>
    [EnumMember(Value = "open-url")]
    OpenUrl = 2,

    /// <summary>
    /// The client will share be able to share their location
    /// </summary>
    [EnumMember(Value = "location-picker")]
    LocationPicker = 3,

    /// <summary>
    /// The client will share be able to share its phone number
    /// </summary>
    [EnumMember(Value = "share-phone")]
    SharePhone = 4,

    /// <summary>
    /// Nothing is sent to the user or account. Just an informative button
    /// </summary>
    [EnumMember(Value = "none")]
    None = 5
}
}
}

```

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2022 р.

**АГРЕГАТОР ЧАТ-БОТІВ ДЛЯ ОПТИМІЗАЦІЇ КОМУНІКАЦІЇ З
КЛІЄНТАМИ З ПЛАТФОРМИ CRM**
Програма та методика тестування
КПІ.ПІ-8215.045430.04.51

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олесь КОВТУНЕЦЬ

Нормоконтроль:

_____ Катерина ЛІЩУК

Виконавець:

_____ Ірина ЗОЗУЛЯ

Київ – 2022

ЗМІСТ

1 Об'єкт випробувань	3
2 Мета тестування	4
3 Методи тестування.....	5
4 Засоби та порядок тестування.....	6

					КП.ІП-8215.045430.04.51	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата		2

1 ОБ'ЄКТ ВИПРОБУВАНЬ

Об'єктом випробування є агрегатор чат-ботів для комунікації, розроблений з використанням технології .Net та мови програмування C#.

					КПІ.ІП-8215.045430.04.51	Арк.
						3
Змін.	Арк.	№ докум.	Підп.	Дата.		

2 МЕТА ТЕСТУВАННЯ

Метою тестування є наступне:

- надійність – здатність програмного забезпечення виконувати задачі протягом встановленого проміжку часу;
- функціональність – визначається здатністю програмного забезпечення вирішувати задачі, що відповідають потребам користувача;
- зручність використання – можливість легкого розуміння, вивчення, використання та привабливості для користувача;
- забезпечення належного рівня безпеки даних;
- зручність взаємодії з ігровим застосуванням;
- знаходження проблем та недоліків з метою їх усунення;
- перевірка зручності графічного інтерфейсу.

					КПІ.ІП-8215.045430.04.51	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		4

3 МЕТОДИ ТЕСТУВАННЯ

Для тестування програмного забезпечення використовуються такі методи:

- статичне тестування – перевіряється уся документація, яка аналізується на предмет дотримання стандартів програмування;
- функціональне тестування, зокрема на рівні Critical path test (базове тестування);
- тестування «білої скриньки» – об'єктом тестування тут є внутрішня поведінка програми. Перевіряється коректність побудови всіх елементів програми та правильність їхньої взаємодії один з одним.

					КП.ІП-8215.045430.04.51	Арк.
						5
Змін.	Арк.	№ докум.	Підп.	Дата.		

4 ЗАСОБИ ТА ПОРЯДОК ТЕСТУВАННЯ

Тестування виконується вручну, з метою знаходження помилок та недоліків як у функціональній частині програмного забезпечення так і в зручності в користуванні.

Для того, щоб перевірити працездатність та відмовостійкість додатку, необхідно провести наступні тестування:

- динамічне тестування на відповідність функціональним вимогам;
- тестування на виведення повідомлень про помилку, коли це необхідно;
- тестування обробки повідомлень, надісланих користувачем;
- тестування обробки повідомлень, надісланих клієнтом;
- тестування інтерфейсу користувача;
- тестування зручності використання.

					КПІ.ІП-8215.045430.04.51	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		6

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ____ ” _____ 2022 р.

**АГРЕГАТОР ЧАТ-БОТІВ ДЛЯ ОПТИМІЗАЦІЇ КОМУНІКАЦІЇ З
КЛІЄНТАМИ З ПЛАТФОРМИ CRM**

Керівництво користувача

КПІ.ІП-8215.045430.05.34

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олесь КОВТУНЕЦЬ

Нормоконтроль:

_____ Катерина ЛІЩУК

Виконавець:

_____ Ірина ЗОЗУЛЯ

Київ – 2022

ЗМІСТ

1 Загальні відомості	3
2 Підготовка до роботи.....	4
3 Робота із застосунком.....	5

					КПІ.ІП-8215.045430.05.34	Арк.
						2
Змін.	Арк.	№ докум.	Підп.	Дата.		

1 ЗАГАЛЬНІ ВІДОМОСТІ

Агрегатор чат-ботів розроблений для зручної комунікації із клієнтами через різні месенджери в одному інтерфейсі. Усі вхідні та вихідні повідомлення одразу відображаються у агрегаторі. Для швидкої навігації між чатами є можливість фільтрувати їх за передналаштованими відборами або пошукового рядка.

Програмне забезпечення реалізоване з використанням тенології .Net мовою програмування C#. Для бази даних було обрано СКБД Microsoft SQL Server. Обмін даними між чат-ботами та агрегатором відбувається за допомогою HTTP-запитів.

					КПІ.ІП-8215.045430.05.34	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		3

2 ПІДГОТОВКА ДО РОБОТИ

Для початку роботи із чат-ботами необхідно спочатку створити боти у необхідних месенджерах, а потім налаштувати їхні параметри на платформі CRM. Усі необхідні налаштування описані у пояснювальній записці «КПІ.ІП-8215.045430.02.81», розділі 4 «Впровадження та супровід програмного забезпечення».

					КПІ.ІП-8215.045430.05.34	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		4

3 РОБОТА ІЗ ЗАСТОСУНКОМ

Інструкцію користувача описано в пояснювальній записці «КПІ.ІП-8215.045430.02.81», розділі 4 «Впровадження та супровід програмного забезпечення».

					КПІ.ІП-8215.045430.05.34	Арк.
Змін.	Арк.	№ докум.	Підп.	Дата.		5

Факультет інформатики та обчислювальної техніки
Кафедра інформатики та програмної інженерії

“ЗАТВЕРДЖЕНО”

Завідувач кафедри

_____ Едуард ЖАРІКОВ

“ ” _____ 2022 р.

**АГРЕГАТОР ЧАТ-БОТІВ ДЛЯ ОПТИМІЗАЦІЇ КОМУНІКАЦІЇ З
КЛІЄНТАМИ З ПЛАТФОРМИ CRM**

Графічний матеріал

КП.ПІ-8215.045430.05.99

“ПОГОДЖЕНО”

Керівник проєкту:

_____ Олесь КОВТУНЕЦЬ

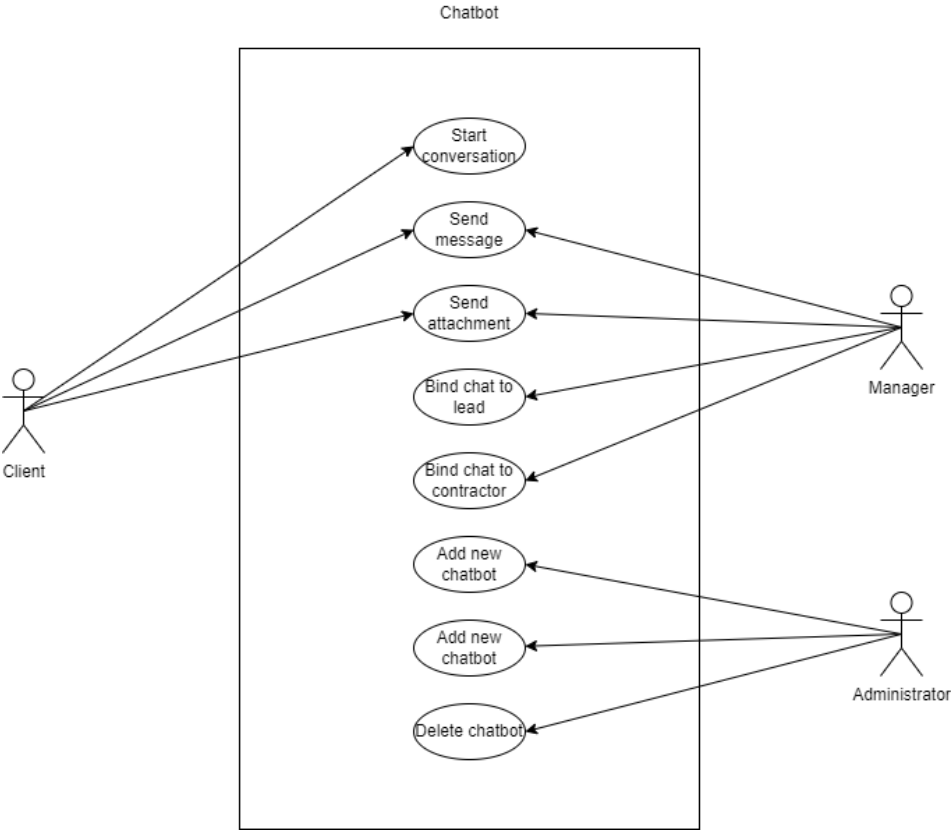
Нормоконтроль:

_____ Катерина ЛІЩУК

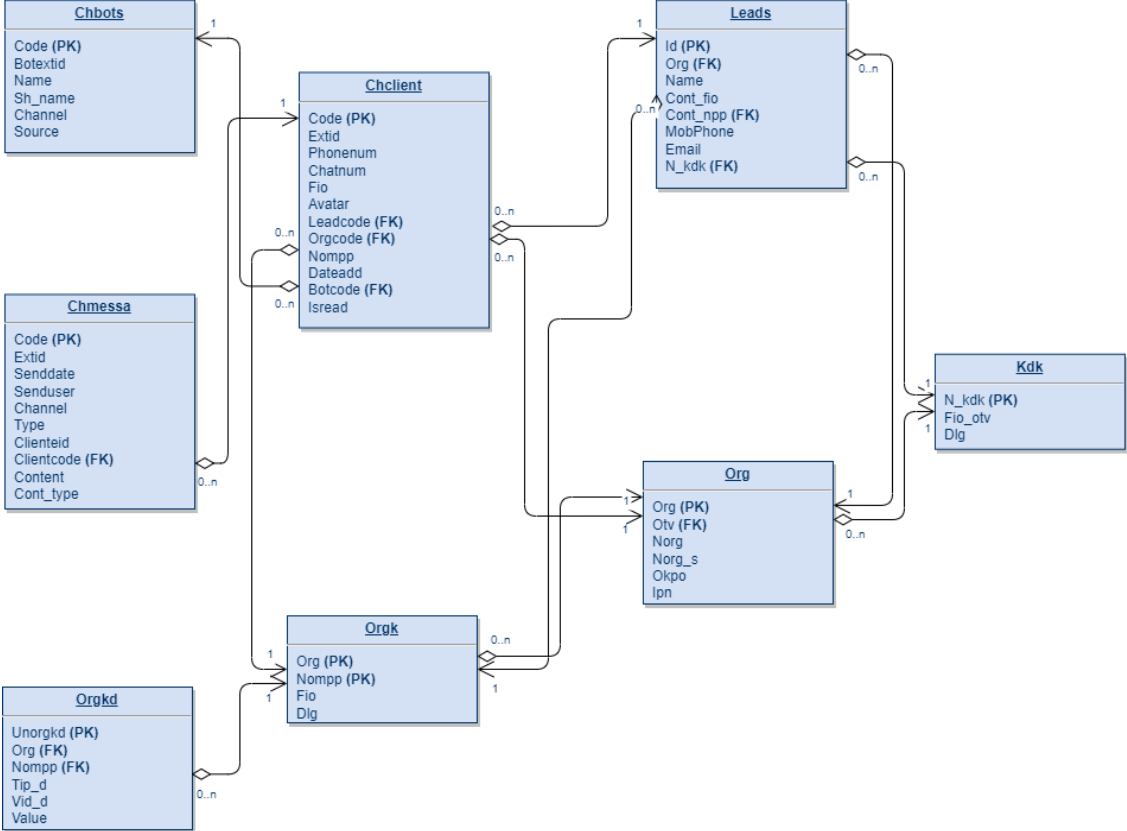
Виконавець:

_____ Ірина ЗОЗУЛЯ

Київ – 2022

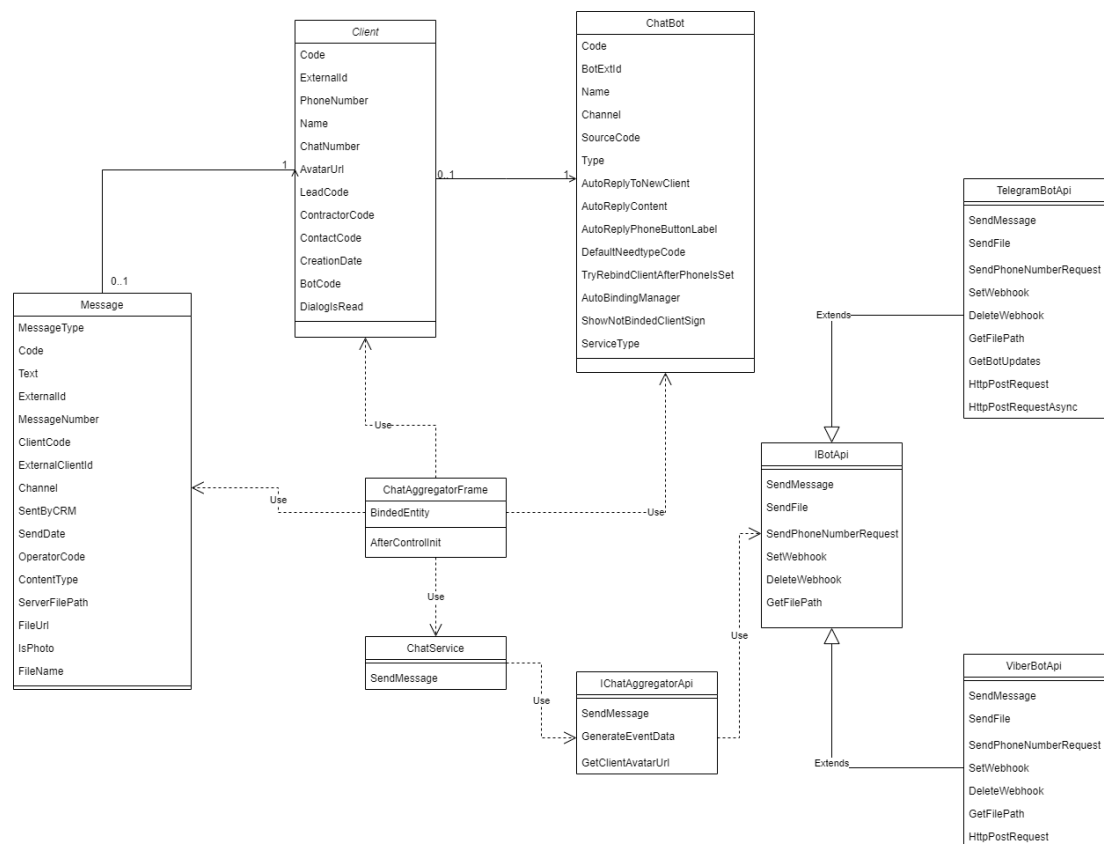


					КПІ.ІІІ-8215.045430.06.99			
Зм.	Арк.	№ докум.	Підпис	Дата				
Розробив		Зозуля І.В.			Схема структурна варіантів використання	Літ.	Арк.	Аркуші
Перевірів		Котунець О.В.					1	1
Реценз.					Агрегатор чат-ботів для оптимізації комунікації з клієнтами з платформи CRM	«КПІ імені Ігоря Сікорського» ФІОТ, гр. ІІІ-82		
Н. Контр.		Ліщук К.І.						
Затв.		Ковтунець О.В.						



					КПІ.ІІІ-8215.045430.08.99			
Зм.	Арк.	№ докум.	Підпис	Дата				
Розробив		Зозуля І.В.			Схема бази даних	Літ.	Арк.	Аркушів
Перевірів		Котунець О.В.					1	1
Реценз.					Агрегатор чат-ботів для оптимізації комунікації з клієнтами з платформи CRM	«КПІ імені Ігоря Сікорського» ФІОТ, гр. ІІІ-82		
Н. Контр.		Ліщук К.І.						
Затв.		Ковтунець О.В.						

КП.ІІІ-8215.045430.09.99.



					КП.ІІІ-8215.045430.09.99			
Зм.	Арк.	№ докум.	Підпис	Дата				
Розробив		Зозуля І.В.			Схема структурна класів програмного забезпечення	Літ.	Арк.	Аркуші
Перевірів		Котунець О.В.					1	1
Реценз.					Агрегатор чат-ботів для оптимізації комунікації з клієнтами з платформи CRM	«КП імені Ігоря Сікорського» ФІОТ, гр. ІІІ-82		
Н. Контр.		Ліщук К.І.						
Затв.		Ковтунець О.В.						

Имя пользователя:
Лісовиченко Олег Іванович

ID проверки:
1011476740

Дата проверки:
06.06.2022 19:43:08 EEST

Тип проверки:
Doc vs Internet + Library

Дата отчета:
06.06.2022 19:44:09 EEST

ID пользователя:
76913

Название файла: IP-82_Зозуля_ПЗ

Количество страниц: 41 Количество слов: 5765 Количество символов: 44866 Размер файла: 1.22 MB ID файла: 1011354484

Обнаружены модификации текста (могут влиять на процент совпадений)

4.98%

Совпадения

Наибольшее совпадение: 1.98% с источником из Библиотеки (ID файла: 1004035523)

1.2% Источники из Интернета 20 Страница 43

4.98% Источники из Библиотеки 211 Страница 43

0% Цитат

Исключение цитат выключено

Исключение списка библиографических ссылок выключено

0% Исключений

Нет исключенных источников

Модификации

Обнаружены модификации текста. Подробная информация доступна в онлайн-отчете.

Замененные символы 2

Подозрительное форматирование 12 страниц